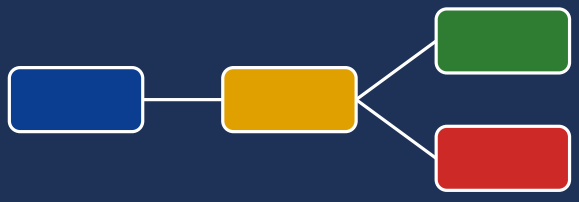
EXECUTION PLANS | WAITS | BLOCKING | PRACTICAL RUNBOOK

Troubleshooting Slow Queries in SQL Server

Execution Plan Analysis and a Practical Runbook

From the first complaint to a verified fix: triage, plan analysis, waits, blocking, resources, client issues, case studies and checklists



WHAT YOU WILL LEARN

- Triage: server, query or application? A 60-second health check
- Finding and reproducing slow queries (Query Store, DMVs, XEvents)
- Reading execution plans: estimates, scans, lookups, joins, spills
- Parameter sniffing, plan regressions, indexes and code patterns
- Waits, blocking, resources, client issues, case studies, checklists

Prepared October 2026 for SQL Server 2016 to 2025. Features differ by version and build; verify details in Microsoft documentation.

Contents

How to Use This Guide . . . 5

1. Understanding 'Slow' . . . 6

1.1 Where the time goes . . . 6

1.2 Types of slow . . . 7

1.3 Ask these questions first . . . 7

2. Triage: Server, Query or Application? . . . 8

2.1 The 60-second health check . . . 8

2.2 Interpreting the health check . . . 9

2.3 Operating-system and platform counters . . . 10

3. Finding the Queries That Matter . . . 11

3.1 Query Store (the best source) . . . 11

3.2 The plan cache and dynamic management views . . . 11

3.3 Extended Events: capture slow statements as they happen . . . 12

3.4 Other sources . . . 13

3.5 Build a baseline . . . 13

4. Reproducing and Measuring a Slow Query . . . 14

4.1 What to capture . . . 14

4.2 Use the same parameters and conditions . . . 14

4.3 'Fast in SSMS, slow in the application' . . . 15

4.4 Find the parameter values behind a bad plan . . . 16

5. Analyzing the Execution Plan . . . 17

5.1 Types of plan and how to get them . . . 17

5.2 Reading order . . . 17

5.3 The ten-step method . . . 18

5.4 A plan read in practice . . . 18

5.5 Operator cheat sheet . . . 19

5.6 Joins at a glance . . . 20

5.7 Seeks, scans and rows read . . . 20

5.8 Warnings to act on . . . 20

6. Estimates, Statistics and Cardinality . . . 21

6.1 Where estimates come from . . . 21

6.2 Why estimates go wrong . . . 21

6.3 Statistics maintenance . . . 22

7. Index Troubleshooting . . . 23

7.1 What the plan tells you about indexes . . . 23
7.2 Designing an index for a query . . . 23
7.3 Seeing which indexes are actually used . . . 23
7.4 Missing index suggestions: use with care . . . 24
7.5 Fragmentation and maintenance: what matters . . . 24
7.6 Special index types . . . 24
8. Parameter Sniffing and Plan Changes . . . 25
8.1 What parameter sniffing is . . . 25
8.2 How to recognise it . . . 25
8.3 Options to fix it . . . 26
8.4 Plan regressions after statistics or version changes . . . 26
9. Query Patterns That Cause Slowness . . . 28
10. Wait Statistics for Troubleshooting . . . 31
10.1 How to use waits . . . 31
10.2 The waits you will meet most often . . . 32
10.3 Waits for one slow query . . . 33
11. Blocking and Deadlocks . . . 34
11.1 Finding the head blocker . . . 34
11.2 Common causes and fixes . . . 34
11.3 Optimistic isolation: RCSI and snapshot . . . 35
11.4 Deadlocks . . . 35
11.5 Handling an incident . . . 35
12. Resource Bottlenecks . . . 36
12.1 CPU . . . 36
12.2 Memory, grants and spills . . . 36
12.3 Disk I/O and storage . . . 37
12.4 tempdb . . . 37
12.5 Transaction log . . . 38
12.6 Parallelism and worker threads . . . 38
13. Client, Network and Application Causes . . . 39
13.1 ASYNC_NETWORK_IO . . . 39
13.2 Application patterns . . . 39
13.3 Linked servers and distributed queries . . . 40
13.4 Network and connectivity checks . . . 40
14. Plan Cache and Compilation Problems . . . 41
14.1 Signs of compile trouble . . . 41

14.2 Fixes for ad hoc plan bloat . . . 41
14.3 Recompilations . . . 41
14.4 Dangerous 'fixes' . . . 42
15. The Tuning Toolbox and Safe Change Process . . . 43
15.1 Choosing the fix . . . 43
15.2 A safe change process . . . 44
15.3 Index changes in production . . . 44
15.4 Using hints responsibly . . . 44
16. Special Scenarios . . . 45
16.1 Slow after an upgrade or compatibility level change . . . 45
16.2 Slow after a restart or failover . . . 45
16.3 Slow at certain times . . . 45
16.4 Slow INSERT, UPDATE and DELETE . . . 46
16.5 Slow reports and analytics . . . 46
16.6 Slow stored procedures: find the slow statement . . . 46
16.7 Slow ETL and bulk loads . . . 47
16.8 Slow in Azure SQL Database and Managed Instance . . . 47
16.9 Slow on a readable secondary or with Availability Groups . . . 47
17. Worked Case Studies . . . 48
17.1 Fast in SSMS, slow in the application . . . 48
17.2 Sudden regression after a statistics update . . . 48
17.3 Key lookup storm . . . 48
17.4 Spills because of a table variable . . . 48
17.5 Blocking from a forgotten transaction . . . 49
17.6 ORM nvarchar parameters and scans . . . 49
17.7 tempdb allocation contention . . . 49
18. Prevention and Monitoring . . . 50
18.1 Turn on and tune the basics . . . 50
18.2 What to alert on . . . 50
18.3 Developer checklist (prevent slow queries at design time) . . . 51
18.4 Upgrades and releases . . . 51
Appendix A: Extra Diagnostic Scripts . . . 52
Appendix B: One-Page Slow Query Checklist . . . 52
Appendix C: Glossary . . . 53
Appendix D: Where to Learn More . . . 53

How to Use This Guide

This guide gives a DBA a repeatable way to troubleshoot slow-running queries in SQL Server, with execution plan analysis at its core. It follows the order you work in during an incident: understand what 'slow' means, triage, find the query, reproduce it, analyse the plan, check waits, blocking and resources, fix it safely and prevent a repeat.

Chapter	You will learn	Use it when you...
1. Understanding 'Slow'	Where time goes, types of slow, the first questions to ask	receive a complaint
2. Triage	Server vs query vs application; 60-second health check	need to know where to start
3. Finding the Queries	Query Store, DMVs, Extended Events, baselines	need the real top offenders
4. Reproducing and Measuring	Same parameters, SET options, fast in SSMS slow in app	cannot reproduce it
5. Analyzing the Plan	Ten-step method, operators, joins, seeks, warnings	have a plan to read
6. Estimates and Statistics	Why estimates fail and how to fix them	see bad row estimates
7. Index Troubleshooting	Index design, missing/unused indexes, maintenance	suspect indexing
8. Sniffing and Plan Changes	Parameter sniffing, regressions, Query Store forcing	it is slow only sometimes
9. Query Patterns	15 common code patterns and fixes	review slow code
10. Wait Statistics	How to use waits; the common waits and first actions	need the bottleneck
11. Blocking and Deadlocks	Find the head blocker, RCSI, deadlock graphs	see LCK_M_* waits
12. Resource Bottlenecks	CPU, memory, I/O, tempdb, log, parallelism	see resource pressure
13. Client and Application	ASYNC_NETWORK_IO, N+1, pooling, linked servers	the database looks fine
14. Plan Cache and Compilation	Compile storms, ad hoc bloat, recompiles	see compile CPU
15. Tuning Toolbox	Choosing fixes and a safe change process	are ready to fix
16. Special Scenarios	After upgrade, restart, ETL, Azure, AGs, procedures	face a specific case
17. Case Studies	Seven worked cases	want examples
18. Prevention	Monitoring, alerts, developer checklist	want fewer incidents

A note on accuracy

Behaviour, DMVs and features differ between SQL Server versions and builds (for example Query Store hints and Parameter Sensitive Plan optimization need SQL Server 2022, OPPO needs 2025). Scripts are provided for orientation: test them on a non-production system first and confirm details in Microsoft's documentation for your release.

1. Understanding 'Slow'

'The query is slow' is a symptom, not a diagnosis. Before looking at any plan, you need to know **what kind of slow** it is, because each kind has a different set of likely causes and a different fastest path to the answer.

1.1 Where the time goes

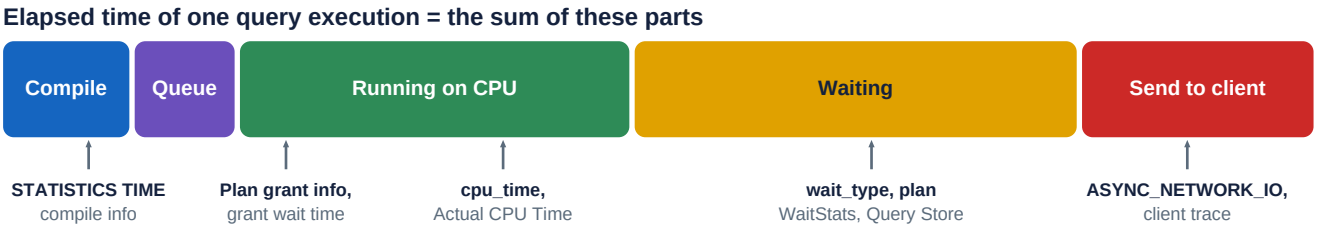


Figure 1.1 - The parts of one query execution and where to measure each

A query can be slow because of *any* of these parts. A query that uses 2 seconds of CPU but takes 40 seconds is mostly **waiting**. A query that finishes in the engine in 1 second but takes 30 seconds to arrive is a **client or network** problem. A query that takes 6 seconds to compile has a **compilation** problem. Measuring the parts first tells you which branch of this guide you need.

Part	How to measure	If it dominates, look at
Compile / parse	SET STATISTICS TIME (parse and compile time); plan properties: Compile Time, Compile CPU; Query Store compile metrics	Plan cache misses, recompiles, very complex queries, optimizer timeouts (chapter 14)
Queue (memory grant, worker thread)	Plan Memory Grant Info / Grant Wait Time; waits RESOURCE_SEMAPHORE, THREADPOOL	Memory grants, concurrency, parallelism (chapters 10, 12)
Running on CPU	cpu_time in sys.dm_exec_requests; STATISTICS TIME CPU time; Actual CPU Time in the plan	Plan shape, scans, functions, parallelism (chapters 5-9)
Waiting on a resource	wait_type / wait_time; Query Store waits; plan WaitStats	Disk, locks, log, latches (chapters 10-12)
Sending results	ASYNC_NETWORK_IO; rows returned; client behaviour	Application and network (chapter 13)

1.2 Types of slow

Pattern	What it usually means	First move
Always slow	Poor plan, missing index, big data volume, inefficient code	Capture the actual plan and read it (chapter 5)
Sometimes slow (same query)	Parameter sniffing, blocking, resource contention at certain times, plan instability	Compare a good and a bad execution: plan, parameters, waits (chapter 8)
Suddenly slow (was fine yesterday)	Plan change after statistics update or restart, data growth, patch, config change, new competing workload	Query Store: did the plan change? What changed on the server? (chapters 8, 16)
Slow in the application, fast in SSMS	Different SET options, parameter values or client-side behaviour	Compare plan cache entries and set_options (chapter 4)
Slow only under load	Concurrency: blocking, memory grants, CPU, tempdb, log, worker threads	Wait statistics during the slow period (chapter 10)
Slow after deployment/upgrade	Compatibility level, new code, dropped index, changed statistics	Compare plans before and after with Query Store (chapters 8, 16)
Slow first time, fast after	Cold cache (data not in memory) or first compile	Check physical reads, restart/failover timing, warm-up (chapter 16)
Slow writes (INSERT/UPDATE/DELETE)	Log latency, many indexes, triggers, blocking, FK checks	Log waits, wide update plans, locks (chapters 6, 11, 12)

1.3 Ask these questions first

Five minutes of good questions often saves hours of tracing. Write the answers in your ticket:

Question	Why it matters
Exactly which operation (screen, report, job, procedure) is slow, and how slow? What is 'normal'?	Defines the target; you cannot fix 'slow' without a goal
When did it start? What changed (code, data volume, statistics, indexes, patches, hardware, settings, users)?	Most regressions have a cause in the last change
Is it slow for everyone, for one user, or for one input value?	Server-wide vs query-specific vs parameter-specific
Does it happen all the time or at certain times (month-end, backups, maintenance)?	Points to contention or scheduled work
Is it slow in SSMS with the same parameters? Which application and driver?	Separates database from client/application issues
How often does it run, and how many rows does it return?	Frequency x cost decides priority; big result sets suggest client issues
What are the SQL Server version, edition, compatibility level and environment (VM, Azure, AG)?	Features and limits differ

Golden rules

Measure before you change. Record duration, CPU, reads and the plan before touching anything.
Change one thing at a time and measure again.
Fix the cause, not the symptom. Restarting, clearing the plan cache or adding hardware may hide the problem for a day.

2. Triage: Server, Query or Application?

Triage decides which of three paths to follow. Run a 60-second health check first so you do not tune a query while the server itself is on fire.

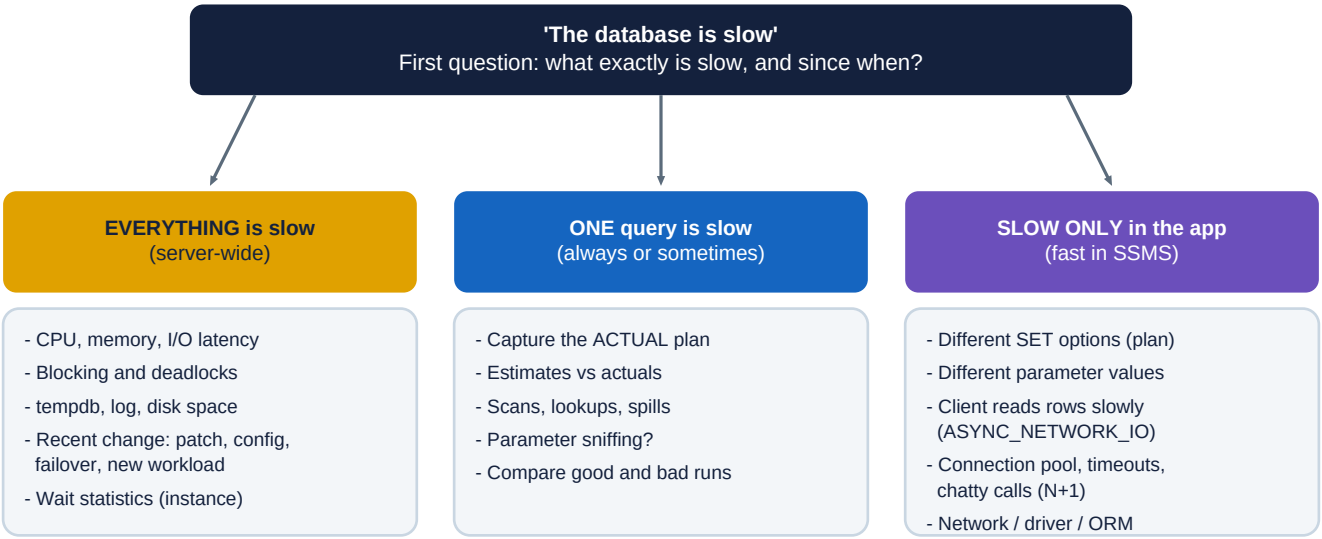


Figure 2.1 - Triage: which path to take

2.1 The 60-second health check

Run these checks in this order. They show whether the problem is server-wide and what is happening right now.

```
-- 1) What is running right now, how long, and what is it waiting for?
SELECT r.session_id, s.login_name, s.host_name, s.program_name, DB_NAME(r.database_id) AS db,
       r.status, r.command, r.total_elapsed_time / 1000.0 AS elapsed_s, r.cpu_time AS cpu_ms,
       r.logical_reads, r.writes, r.wait_type, r.wait_time AS wait_ms, r.blocking_session_id,
       r.granted_query_memory * 8 / 1024 AS grant_mb, r.dop,
       SUBSTRING(t.text, r.statement_start_offset / 2 + 1,
                 (CASE r.statement_end_offset WHEN -1 THEN DATALENGTH(t.text) ELSE
                     r.statement_end_offset END
                  - r.statement_start_offset) / 2 + 1) AS current_statement
FROM sys.dm_exec_requests r
JOIN sys.dm_exec_sessions s ON s.session_id = r.session_id
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE s.is_user_process = 1 AND r.session_id <> @@SPID
ORDER BY r.total_elapsed_time DESC;
```

```
-- 2) Is anyone blocked, and who is the head blocker?
SELECT r.session_id AS blocked, r.blocking_session_id AS blocker, r.wait_type, r.wait_time / 1000.0 AS
    wait_s, r.wait_resource
FROM sys.dm_exec_requests r WHERE r.blocking_session_id <> 0 ORDER BY r.wait_time DESC;

-- 3) Waiting tasks right now (includes parallel threads)
SELECT session_id, exec_context_id, wait_type, wait_duration_ms, blocking_session_id,
    resource_description
FROM sys.dm_os_waiting_tasks WHERE session_id > 50 ORDER BY wait_duration_ms DESC;

-- 4) Oldest open transactions (a forgotten BEGIN TRAN blocks and bloats the log)
SELECT TOP 10 s.session_id, t.transaction_begin_time, DATEDIFF(MINUTE, t.transaction_begin_time,
    GETDATE()) AS minutes_open
FROM sys.dm_tran_active_transactions t JOIN sys.dm_tran_session_transactions s ON s.transaction_id =
    t.transaction_id
ORDER BY t.transaction_begin_time;
```

```
-- 5) CPU queue and worker shortage
SELECT scheduler_id, current_tasks_count, runnable_tasks_count, work_queue_count FROM
    sys.dm_os_schedulers WHERE scheduler_id < 255;

-- 6) Disk latency per file since startup (high numbers on data or log files point to storage)
SELECT DB_NAME(vfs.database_id) AS db, mf.type_desc, mf.physical_name,
       vfs.io_stall_read_ms / NULLIF(vfs.num_of_reads, 0) AS avg_read_ms,
       vfs.io_stall_write_ms / NULLIF(vfs.num_of_writes, 0) AS avg_write_ms
FROM sys.dm_io_virtual_file_stats(NULL, NULL) vfs JOIN sys.master_files mf ON mf.database_id =
    vfs.database_id AND mf.file_id = vfs.file_id
ORDER BY vfs.io_stall DESC;

-- 7) Memory grants waiting
SELECT session_id, requested_memory_kb, granted_memory_kb, wait_time_ms FROM
    sys.dm_exec_query_memory_grants WHERE granted_memory_kb IS NULL;
```

2.2 Interpreting the health check

You see	It suggests	Go to
Many sessions blocked by one head blocker	Blocking caused by a long transaction or a lock-heavy statement	Chapter 11
Many sessions in runnable state; runnable_tasks_count > 0 on all schedulers; Windows CPU near 100%	CPU pressure (heavy queries, too much parallelism, compile storms)	Chapters 10, 12
Waits dominated by PAGEIOLATCH_*, IO_COMPLETION, or file latency above 10-20 ms	Data I/O pressure	Chapter 12
WRITELOG / LOGBUFFER waits, log file latency above a few ms	Transaction log bottleneck	Chapter 12
RESOURCE_SEMAPHORE waits, grants not granted	Memory grant queue	Chapter 12
THREADPOOL waits; new connections time out	Worker thread exhaustion	Chapters 11, 12
PAGELATCH_* on tempdb pages	tempdb allocation contention	Chapter 12
One session in ASYNC_NETWORK_IO	Client not consuming results	Chapter 13
Nothing unusual; one query is simply slow	Query-specific problem	Chapters 4-9

2.3 Operating-system and platform counters

Counter	Rough guidance	Meaning
Processor: % Processor Time	Sustained above 80-85% needs attention	CPU saturation
Physical Disk: Avg. Disk sec/Read and sec/Write	Data files: under about 10-20 ms; log: under about 1-5 ms	Storage latency
SQLServer:Buffer Manager: Page life expectancy	Judge against your baseline; sudden drops matter more than a fixed number	How long pages stay cached
SQLServer:Buffer Manager: Page reads/sec	Compare to baseline	Physical read pressure
SQLServer:Memory Manager: Memory Grants Pending	Should be 0	Queries waiting for memory
SQLServer:SQL Statistics: Batch Requests/sec, SQL Compilations/sec, SQL Re-Compilations/sec	Compilations should be a small fraction of batch requests	Workload volume and compile churn
SQLServer:Locks: Lock Waits/sec, Number of Deadlocks/sec	Deadlocks should be near 0	Blocking and deadlocks
SQLServer:Access Methods: Full Scans/sec, Page Splits/sec	Compare to baseline	Scan-heavy workloads, index maintenance side effects
SQLServer:General Statistics: User Connections, Processes blocked	Watch for spikes	Connection storms and blocking
Hypervisor: CPU ready/steal, ballooning	Any sustained value is a red flag in VMs	Resources promised but not delivered

Do not skip the platform

A slow query is sometimes a healthy query on an unhealthy platform: a saturated virtual host, a degraded storage path, antivirus scanning data files, a power plan set to 'balanced', or a full disk. If many unrelated queries slowed down at the same time, look at the platform first.

3. Finding the Queries That Matter

Tuning the loudest complaint is not always tuning the biggest problem. Use data to rank queries by **total impact**: cost per execution multiplied by how often it runs.

3.1 Query Store (the best source)

Query Store keeps the history of queries, plans, runtime statistics and (in recent versions) wait statistics inside each database. It survives restarts and plan cache clears, so it answers 'what changed?'. It is on by default for new databases in SQL Server 2022 and later; enable it elsewhere.

```
ALTER DATABASE [SalesDB] SET QUERY_STORE = ON
  (OPERATION_MODE = READ_WRITE, QUERY_CAPTURE_MODE = AUTO, MAX_STORAGE_SIZE_MB = 2048,
   INTERVAL_LENGTH_MINUTES = 30, WAIT_STATS_CAPTURE_MODE = ON, CLEANUP_POLICY =
       (STALE_QUERY_THRESHOLD_DAYS = 30));
```

SSMS Query Store report	Use it to
Top Resource Consuming Queries	Rank by duration, CPU, reads, writes, memory, tempdb, executions; see plans per query
Regressed Queries	Find queries whose recent duration is worse than before and compare plans
Overall Resource Consumption	See server-level trends and spikes by time
Queries With Forced Plans / High Variation	Review forced plans; find queries with unstable performance (sniffing candidates)
Query Wait Statistics	See which wait categories dominate which queries
Tracked Queries	Follow one query over time

```
-- Top queries by total duration in the last 24 hours (frequency x cost)
SELECT TOP 20 q.query_id, MIN(qt.query_sql_text) AS sql_text, SUM(rs.count_executions) AS executions,
       SUM(rs.avg_duration * rs.count_executions) / 1000000.0 AS total_duration_s,
       SUM(rs.avg_cpu_time * rs.count_executions) / 1000000.0 AS total_cpu_s,
       SUM(rs.avg_logical_io_reads * rs.count_executions) AS total_reads
FROM sys.query_store_query q
JOIN sys.query_store_query_text qt ON qt.query_text_id = q.query_text_id
JOIN sys.query_store_plan p ON p.query_id = q.query_id
JOIN sys.query_store_runtime_stats rs ON rs.plan_id = p.plan_id
JOIN sys.query_store_runtime_stats_interval i ON i.runtime_stats_interval_id =
    rs.runtime_stats_interval_id
WHERE i.start_time > DATEADD(HOUR, -24, SYSUTCDATETIME())
GROUP BY q.query_id ORDER BY total_duration_s DESC;
```

3.2 The plan cache and dynamic management views

When Query Store is off, use the plan cache. It only contains plans still in memory, so recent restarts or memory pressure empty it. Remember that totals accumulate since the plan was cached.

```
-- Top 15 statements by CPU, with per-execution averages and variation (max vs avg suggests sniffing)
SELECT TOP 15 qs.execution_count,
       qs.total_worker_time / qs.execution_count / 1000 AS avg_cpu_ms, qs.max_worker_time / 1000 AS
           max_cpu_ms,
       qs.total_elapsed_time / qs.execution_count / 1000 AS avg_ms, qs.max_elapsed_time / 1000 AS
           max_ms,
       qs.total_logical_reads / qs.execution_count AS avg_reads, qs.max_logical_reads AS max_reads,
       qs.creation_time, qs.last_execution_time,
       SUBSTRING(st.text, qs.statement_start_offset / 2 + 1,
                 (CASE qs.statement_end_offset WHEN -1 THEN DATALENGTH(st.text) ELSE
                     qs.statement_end_offset END
                  - qs.statement_start_offset) / 2 + 1) AS statement_text,
       qp.query_plan
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle) qp
ORDER BY qs.total_worker_time DESC;
```

Rank by	Finds	Typical culprit
total_worker_time (CPU)	Queries burning CPU	Scans, functions, bad plans, executions that are cheap but too frequent
total_logical_reads	Queries doing the most page reads	Missing indexes, lookups, scans; pressure on memory and I/O
total_elapsed_time	Queries that take longest overall	Waits: blocking, I/O, grants, client
execution_count	Chatty queries	Application loops (N+1), missing caching
max vs average (elapsed, reads)	Unstable queries	Parameter sniffing, data skew
total_grant_kb / max_spills (when available)	Memory-hungry or spilling queries	Sorts, hashes, bad estimates

3.3 Extended Events: capture slow statements as they happen

```
-- Lightweight session: statements slower than 5 seconds (duration is in microseconds)
CREATE EVENT SESSION [slow_queries] ON SERVER
ADD EVENT sqlserver.sql_batch_completed (ACTION (sqlserver.sql_text, sqlserver.database_name,
    sqlserver.client_app_name,
                                                sqlserver.client_hostname, sqlserver.username) WHERE
                                                    duration > 5000000),
ADD EVENT sqlserver.rpc_completed       (ACTION (sqlserver.sql_text, sqlserver.database_name,
    sqlserver.client_app_name,
                                                sqlserver.client_hostname, sqlserver.username) WHERE
                                                    duration > 5000000)
ADD TARGET package0.event_file (SET filename = N'slow_queries.xel', max_file_size = (50),
    max_rollover_files = (4))
WITH (MAX_DISPATCH_LATENCY = 5 SECONDS, STARTUP_STATE = OFF);
ALTER EVENT SESSION [slow_queries] ON SERVER STATE = START;
```

Be careful with plan-capturing events

Events such as query_post_execution_showplan capture actual plans but can slow a busy server substantially. Use narrow filters (one database, long duration, a specific query hash), run briefly and stop the session.

3.4 Other sources

- **Activity Monitor** and **Performance Dashboard reports** in SSMS for a quick look at recent expensive queries and waits.
- **SQL Server Agent job history** for batch jobs whose duration has crept up.
- **Application logs and APM tools** for end-to-end timings, including time spent outside the database.
- **Community tools** that wrap these DMVs (for example sp_WhoIsActive and the First Responder Kit procedures) are popular and save time; review the code and test before using them on production.

3.5 Build a baseline

'Slow' is relative to normal. Keep a baseline of a healthy period: top waits, top queries, file latency, CPU, batch requests/sec and typical durations for key operations. Query Store plus a scheduled snapshot of `sys.dm_os_wait_stats` and file stats into a small table gives you one at almost no cost.

4. Reproducing and Measuring a Slow Query

You cannot fix what you cannot reproduce. The goal is to run the **same statement, with the same parameters and settings** as the slow case, and capture numbers you can compare after the fix.

4.1 What to capture

Measurement	How	What it tells you
Actual execution plan	SSMS 'Include Actual Execution Plan'; SET STATISTICS XML ON	Real row counts, memory grant, spills, waits, parameter values
Logical reads per table	SET STATISTICS IO ON	Data access cost; the most stable metric for before/after comparison
CPU and elapsed time, compile time	SET STATISTICS TIME ON	Time spent compiling vs running; CPU vs waiting
Rows returned and result size	Messages, client	Large results point to client/network issues
Waits during the run	Plan WaitStats; sys.dm_exec_session_wait_stats; Query Store	Which resource it waited on

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
SET STATISTICS XML ON;               -- or use the SSMS actual plan button
EXEC dbo.GetCustomerOrders @CustomerID = 1, @From = '2026-01-01';
SET STATISTICS XML OFF; SET STATISTICS TIME OFF; SET STATISTICS IO OFF;
-- Messages tab example:  Table 'Orders'. Scan count 1, logical reads 48211, physical reads 0,
    read-ahead reads 0 ...
-- Compare logical reads and CPU time before and after each change.
```

4.2 Use the same parameters and conditions

- **Parameter values:** get the slow run's real values from Query Store, an Extended Events capture, the application log, or the plan's ParameterList (compiled value and runtime value). Test with the *slow* value and with a *typical* value.
- **Data state:** use the same database (or a recent restore). Small test data hides scan and spill problems.
- **Cache state:** the first run reads from disk (cold) and later runs from memory (warm). Run twice and note both. On a test server, `DBCC DROPCLEANBUFFERS` forces cold runs; **never run it or DBCC FREEPROCCACHE on production**.
- **Concurrency:** a query that is fast alone can be slow under load because of blocking or memory grants. Reproduce under load if needed.
- **Security context:** row-level security, different logins or schemas can change plans.

4.3 'Fast in SSMS, slow in the application'

This is one of the most common support cases. The usual reason is that **the application and SSMS use different cached plans**. A cached plan is keyed by the query text *and* several session SET options. SSMS turns ARITHABORT ON by default; many drivers leave it OFF. Same text, different options, separate plan cache entries, each compiled for whichever parameter values arrived first.

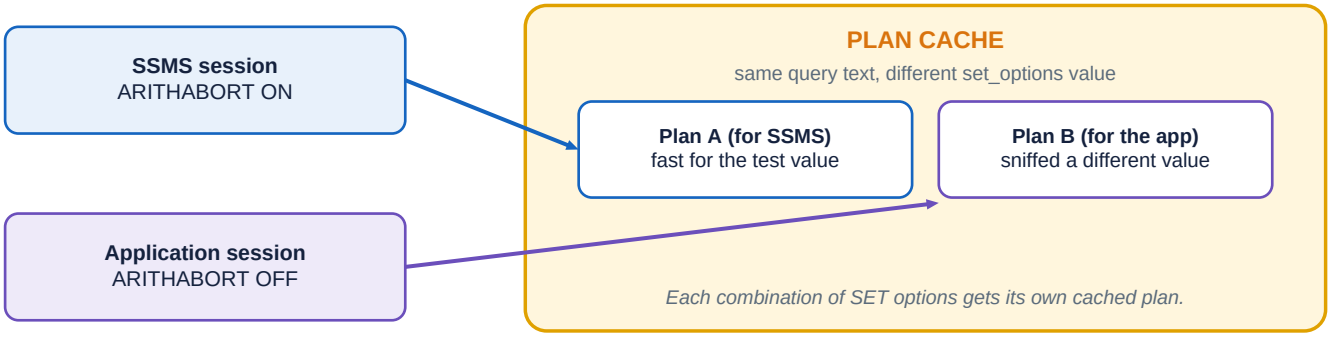


Figure 4.1 - SSMS and the application can be using different cached plans for the same query

```
-- Which plan cache entries exist for this query, and what SET options does each use?
SELECT cp.plan_handle, cp.usecounts, cp.size_in_bytes / 1024 AS plan_kb,
       CAST(pa.value AS int) AS set_options,
       CASE WHEN CAST(pa.value AS int) & 4096 = 4096 THEN 'ARITHABORT ON' ELSE 'ARITHABORT OFF' END AS
           arithabort,
       st.text
FROM sys.dm_exec_cached_plans cp
CROSS APPLY sys.dm_exec_sql_text(cp.plan_handle) st
CROSS APPLY sys.dm_exec_plan_attributes(cp.plan_handle) pa
WHERE pa.attribute = 'set_options' AND st.text LIKE '%GetCustomerOrders%' AND st.text NOT LIKE
    '%dm_exec_cached_plans%';
```

```
-- Reproduce the application's plan in SSMS: use the same SET options as the application
SET ARITHABORT OFF;            -- many drivers default to OFF; check the actual session options of the
    app
EXEC dbo.GetCustomerOrders @CustomerID = 1, @From = '2026-01-01';

-- Check what the application session really uses
SELECT session_id, program_name, arithabort, ansi_nulls, ansi_padding, ansi_warnings,
    quoted_identifier, concat_null_yields_null
FROM sys.dm_exec_sessions WHERE program_name LIKE '%YourApp%';
```

Cause	Test	Remedy
Different SET options (plan cache entry)	Run with the app's options; compare plans and cache entries	Fix the real problem in the plan (stats, sniffing, index); align connection settings where sensible
Different parameter values	Use the application's actual values	Treat as parameter sniffing (chapter 8)
Different data type of parameters (ORM sends nvarchar)	Compare parameter types in the trace/plan	Fix parameter types (chapter 9)
Client slow at reading results	Check ASYNC_NETWORK_IO and rows returned	Return fewer rows; fix client (chapter 13)
Different connection (network, driver, MARS, encryption)	Test from the app server with sqlcmd	Network and driver investigation
Different isolation level or transaction handling	Compare session settings	Fix transaction scope; consider RCSI

Do not 'fix' it with ARITHABORT
Setting ARITHABORT in the application only makes the two cache entries identical; it hides the symptom. If the shared plan is bad for some values, the real cause is parameter sensitivity, estimates or missing indexes.

4.4 Find the parameter values behind a bad plan

- In the plan XML or the graphical plan, select the root operator and open **Parameter List**: *Parameter Compiled Value* is what the plan was built for; *Parameter Runtime Value* (actual plans) is what was used in that run.
- In Query Store, open the plan XML. For the real values of slow runs capture them with an Extended Events session on `rpc_completed`; the statement and parameters are in the event data.
- For ad hoc application SQL, literal values are already in the statement text.

5. Analyzing the Execution Plan

The execution plan shows *how* SQL Server ran your query. Once you can read it, most slow queries explain themselves: the plan shows where rows explode, where data is read wastefully and where work is thrown away.

5.1 Types of plan and how to get them

Plan	Contains	Get it with
Estimated	Compiled plan and estimates; query is not run	SSMS Display Estimated Plan; SET SHOWPLAN_XML ON; plan cache; Query Store
Actual	Same plan plus real rows, executions, time, memory grant, waits, spills	SSMS Include Actual Plan; SET STATISTICS XML ON
Last known actual	Actual counters of the last execution of a cached plan (SQL Server 2019+)	sys.dm_exec_query_plan_stats with LAST_QUERY_PLAN_STATS = ON
Live (in flight)	Progress of a running query	SSMS Live Query Statistics; sys.dm_exec_query_statistics_xml

Estimated vs actual

The actual plan has the **same shape and the same estimates** as the estimated plan; it is not re-optimised. The actual numbers are added so that you can compare them with the guesses. A plan built on wrong guesses is the usual reason for a slow query.

5.2 Reading order

The graphical plan is a tree. Arrows point **right to left** (the way rows flow) and thicker arrows mean more rows. Execution is pulled by the root on the left. Top inputs of joins are the outer/build side; bottom inputs are the inner/probe side.

Top input of a join (outer) is drawn above; bottom input (inner) below.

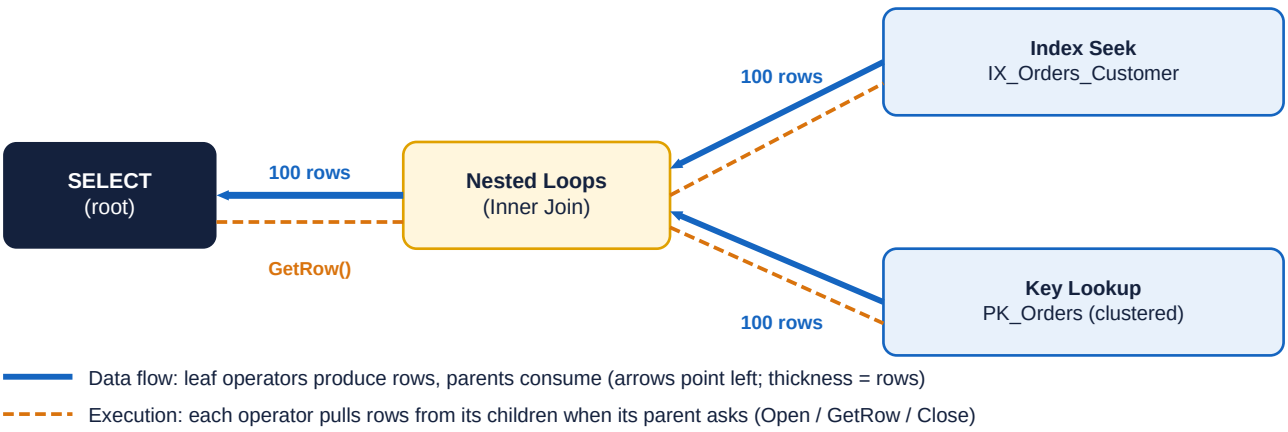
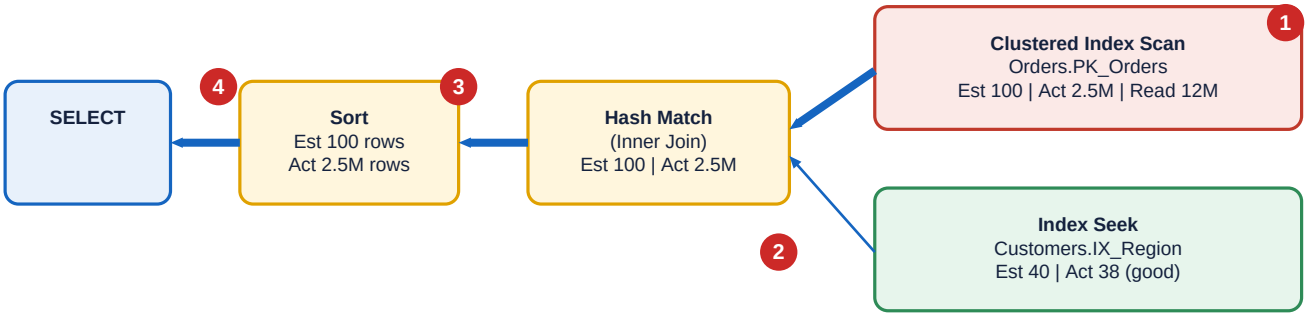


Figure 5.1 - Data flows right to left; the root pulls rows

5.3 The ten-step method

Step	Look at	Questions
1	Root operator properties	Compile time? Optimization early abort? DOP? Memory grant and wait? Compiled vs runtime parameter values? SET options?
2	Actual time and rows	Which operators take the time (Actual Elapsed/CPU)? Where are the biggest row counts and thickest arrows?
3	Estimated vs actual rows	Differences of 10x or more? Which operator first goes wrong?
4	Data flow	Where do rows explode (bad join) or get thrown away (late filter)?
5	Access methods	Scan or seek? Rows Read vs Actual Rows? Residual Predicate? Key Lookups and how many executions?
6	Joins	Join type appropriate for row counts? Inner executions? Many-to-many merge? Hash warning?
7	Sorts, spools, spills	Sort Warning/Hash Warning? Eager spool? Huge or tiny memory grants?
8	Parallelism	DOP? Skewed threads? Non Parallel Plan Reason?
9	Warnings and missing index	Implicit conversion? No join predicate? Missing statistics?
10	STATISTICS IO/TIME and WaitStats	Which table dominates reads? CPU vs elapsed? Top waits?

5.4 A plan read in practice



1. Scan reads 12M rows to return 2.5M: no useful index on the filter column, or a non-SARGable predicate.
2. Estimate 100 vs actual 2.5M (25,000x): stale statistics, parameter sniffing or a poor predicate estimate.
3. Hash Match got a memory grant sized for 100 rows, so it spilled to tempdb (Hash Warning).
4. Sort also spilled; the same wrong estimate gave it a tiny memory grant.

Figure 5.2 - A plan with a wrong estimate that causes a scan, a hash join and two spills

Diagnosis: the optimizer expected about 100 orders but 2.5 million qualified, so it chose a scan feeding a hash join and a sort with tiny memory grants that spilled to tempdb. The root cause is the **estimate**. Fix the estimate (statistics, parameters, predicate shape) and the grant, join and spills usually fix themselves. The scan itself says a better index may also be needed.

5.5 Operator cheat sheet

Operator	What it does	Fine when	Trouble when
Clustered Index Seek / Index Seek	Navigates the B-tree to matching keys	Selective predicate on the leading key	Returns millions of rows; residual predicate removes most
Clustered Index Scan / Index Scan / Table Scan	Reads the whole index or heap	Small table or most rows needed	Rows Read far larger than Actual Rows
Key Lookup / RID Lookup	Fetches missing columns per row from the table	Few executions	Tens of thousands of executions; add covering columns
Nested Loops	Runs the inner side once per outer row	Small outer, indexed inner	Large outer or inner scan; estimate of 1 execution
Merge Join	Walks two sorted inputs together	Presorted inputs, one-to-many	Sorts added below; many-to-many
Hash Match (join)	Hash table from build input, probe with other	Large unsorted inputs, in memory	Spill, wrong build side, huge grant
Adaptive Join	Chooses loop or hash at run time	Uncertain row counts, batch mode	Large grant for a loop outcome
Hash Match (aggregate) / Stream Aggregate	Group and aggregate (hash vs sorted input)	Moderate group counts / ordered input	Spill / a Sort feeding Stream Aggregate
Sort / TopN Sort	Orders rows	Small inputs or index-ordered	Spill, millions of rows, wide rows
Compute Scalar	Evaluates expressions	Cheap expressions	Hides a scalar function called per row
Filter	Removes rows not removed earlier	Early and cheap	Late filter after expensive joins
Table / Index Spool	Saves intermediate rows in tempdb	Rewinds reuse rows	Rebinds rebuild it; usually a missing index
Parallelism (Gather/Repartition/Distribute)	Moves rows between threads	Even distribution	Skew, order-preserving merge, exchange spill
Table-valued function	Evaluates a multi-statement TVF into tempdb	Rare	Fixed 1/100-row estimate; replace with inline TVF
Remote Query / Remote Scan	Queries a linked server	Filtered remotely	Pulls large remote data
Insert / Update / Delete (Clustered, Index)	Modify rows and indexes	Narrow plan, few indexes	Wide plan, many indexes, triggers, huge batches
Assert	Checks a constraint or single-row subquery	Constraint checks	Scalar subquery returning many rows
Window Spool	Expands window frames	ROWS frame in memory	Default RANGE frame on disk

5.6 Joins at a glance

Join	Needs	Memory	Best for	Typical failure
Nested Loops	Index on inner join column helps	None	Few outer rows (OLTP lookups)	Outer underestimated; inner scan
Merge Join	Both inputs sorted on join keys	None unless Sort added	Large sorted inputs	Sorts, many-to-many, parallel deadlock
Hash Match	Equality predicate	Grant for hash table	Large unsorted inputs, analytics	Spill (grant too small) or giant grant
Adaptive Join	Batch mode, eligible join	Grant for hash option	Uncertain estimates	Grant sized for the hash side

5.7 Seeks, scans and rows read

Observation	Meaning	Action
Index Seek, Seek Predicate on first key column	Efficient navigation	Usually good
Seek with a large residual Predicate	Index only narrows by one column; the rest is row-by-row filtering	Reorder or extend the index so more predicates become seek predicates
Scan with Number of Rows Read much larger than Actual Rows	Reading rows only to discard them	Add an index on the filter columns; make the predicate SARGable
Seek returning most of the table	The range is not selective	A scan plus hash join may be better; reconsider design
Key Lookup executed 100,000 times	Each row costs a random read	Covering index (INCLUDE) or fewer columns

SARGable predicates

Keep the indexed column **bare** in the predicate. `WHERE YEAR(OrderDate) = 2026` cannot seek; `WHERE OrderDate >= '2026-01-01' AND OrderDate < '2027-01-01'` can.

5.8 Warnings to act on

Warning	Meaning	Fix
Implicit conversion (PlanAffectingConvert)	A conversion on a column prevents seeks or distorts estimates	Match parameter/literal types to the column
Sort / Hash warning (spill)	Operator used more memory than granted and wrote to tempdb	Fix estimates; reduce rows/width; add index for order
Excessive or insufficient memory grant	Grant much larger or smaller than used	Fix estimates; feedback features; grant hints
No join predicate	Cross product	Add the missing condition
Columns with no statistics	Optimizer guessed	Create or enable statistics
Missing index	Optimizer suggests an index	Evaluate carefully (chapter 7)
Optimizer timeout / memory limit	Search stopped early; plan may be poor	Simplify the query
Parameter value mismatch	Compiled value differs from runtime value	Parameter sniffing (chapter 8)

6. Estimates, Statistics and Cardinality

A plan is only as good as its row estimates. When the optimizer thinks 100 rows will come out and 5 million do, every later choice (join type, memory, parallelism) is wrong.

6.1 Where estimates come from

- **Statistics** on columns and indexes: a histogram (up to 200 steps) of the first column's values, plus density (how unique) of column combinations.
- **The cardinality estimator** combines them: equality on a known value uses the histogram; an unknown value (local variable) uses average density; several predicates are combined with assumptions (independence in the legacy estimator, exponential backoff in the newer one at compatibility level 120+).
- **Special cases:** table variables (1 row before SQL Server 2019 compatibility 150), multi-statement table functions (fixed guess before compatibility 140), ascending key values newer than the last histogram step.

```
DBCC SHOW_STATISTICS ('dbo.Orders', 'IX_Orders_OrderDate');       -- header, density vector, histogram

SELECT OBJECT_NAME(s.object_id) AS tbl, s.name AS stat, sp.last_updated, sp.rows, sp.rows_sampled,
    sp.modification_counter
FROM sys.stats s CROSS APPLY sys.dm_db_stats_properties(s.object_id, s.stats_id) sp
WHERE OBJECTPROPERTY(s.object_id, 'IsUserTable') = 1 AND sp.modification_counter > 0
ORDER BY sp.modification_counter DESC;                             -- most-changed statistics first
```

6.2 Why estimates go wrong

Cause	Plan clue	Fix
Stale or low-sample statistics	Estimates off on many operators of one table	UPDATE STATISTICS (FULLSCAN or larger sample); update more often for volatile tables
Parameter sniffing	Compiled value differs from runtime value	See chapter 8
Local variables / unknown values	Estimate equals average density, ignores the value	Use parameters, OPTION (RECOMPILE), or dynamic SQL
Correlated columns	Estimate too low after several ANDed predicates	Multi-column or filtered statistics; newer CE
Skewed data	Average hides hot values	Higher sample, filtered statistics, PSP optimization
Functions/expressions on columns	Guess instead of histogram	Rewrite SARGable; computed column with index
Implicit conversion	CONVERT_IMPLICIT in predicate	Fix data types
Table variables / MSTVFs	Estimate of 1 or 100 rows	Temp tables; inline TVFs; compatibility 140/150+
Ascending key beyond histogram	Estimate near 1 for newest data	Update statistics frequently on those columns
Many joins, nested views	Errors multiply through the plan	Break into steps with temp tables; simplify; index join columns

6.3 Statistics maintenance

- Leave **AUTO_CREATE_STATISTICS** and **AUTO_UPDATE_STATISTICS** on. Consider **AUTO_UPDATE_STATISTICS_ASYNC** only after testing (the query that triggers the update uses the old statistics).
- Auto update triggers after roughly 500 rows + 20% of the table changed (legacy), or a lower dynamic threshold for large tables at compatibility 130+. For big, fast-changing tables schedule extra updates.
- **Index rebuilds** update statistics with a full scan; **reorganize does not**. Do not rebuild indexes just to update statistics; use UPDATE STATISTICS.
- For critical tables use **FULLSCAN** or persisted sample percent so that skew is captured.
- Updating statistics causes affected queries to **recompile**, which can change plans; this is why a plan can change 'overnight'.

> **Rule of thumb**
> An estimate within 10x of reality rarely changes the plan. An error of 100x or more almost always does. Always find the **first operator where estimate and actual diverge**: that is where the problem starts.

7. Index Troubleshooting

Indexes are the most powerful and the most overused tuning tool. The goal is not more indexes, but the **right few** for your queries.

7.1 What the plan tells you about indexes

Plan clue	Possible index action
Scan reading far more rows than returned	Index on the filter column(s); make predicate SARGable
Seek + Key Lookup with many executions	Covering index: add needed columns with INCLUDE
Sort feeding a Merge Join, Stream Aggregate or ORDER BY	Index in the sort order (check ASC/DESC)
Hash Match on two large scans	Indexes on join keys may allow seek or merge
Index Spool	The optimizer built a temporary index: create the permanent one
Missing index green text	Candidate only; validate (section 7.4)
Scan on huge table for analytics	Columnstore index (batch mode, segment elimination)

7.2 Designing an index for a query

- **Equality predicates first** (columns compared with =), most selective first when otherwise equal.
- **Then the range or sort column** (one column compared with >, <, BETWEEN, or the ORDER BY column), because only the first range column can be seeked.
- **INCLUDE** the other columns that the query selects, so no lookups are needed; keep includes lean.
- **Match ORDER BY and GROUP BY** to the key order so that sorts disappear.
- **Filtered indexes** for queries that always filter a small subset (for example Status = 'Open').
- **Do not index low-selectivity columns alone** (a bit column alone rarely helps).
- Consider the **write cost**: every index is updated on INSERT/UPDATE/DELETE and takes space and memory.

```
-- Query:  customer's orders in a date range, newest first
SELECT OrderID, OrderDate, Status, TotalAmount
FROM dbo.Orders WHERE CustomerID = @CustomerID AND OrderDate >= @From AND OrderDate < @To
ORDER BY OrderDate DESC;

-- Index: equality (CustomerID), then range/sort (OrderDate), include the rest (no key lookups, no sort)
CREATE INDEX IX_Orders_Customer_Date ON dbo.Orders (CustomerID, OrderDate DESC) INCLUDE (Status,
    TotalAmount);
```

7.3 Seeing which indexes are actually used

```
-- Unused or write-only indexes since last restart (verify over a full business cycle before dropping)
SELECT OBJECT_NAME(i.object_id) AS tbl, i.name AS index_name, i.type_desc,
       ISNULL(us.user_seeks, 0) AS seeks, ISNULL(us.user_scans, 0) AS scans, ISNULL(us.user_lookups, 0)
           AS lookups, ISNULL(us.user_updates, 0) AS updates
FROM sys.indexes i
LEFT JOIN sys.dm_db_index_usage_stats us ON us.object_id = i.object_id AND us.index_id = i.index_id AND
    us.database_id = DB_ID()
WHERE OBJECTPROPERTY(i.object_id, 'IsUserTable') = 1 AND i.index_id > 1 AND i.is_primary_key = 0 AND
    i.is_unique_constraint = 0
ORDER BY ISNULL(us.user_seeks, 0) + ISNULL(us.user_scans, 0) + ISNULL(us.user_lookups, 0),
    us.user_updates DESC;
```

7.4 Missing index suggestions: use with care

- The suggestion is based on **one query** and ignores existing similar indexes, index size and write cost. It may also propose very wide includes.
- Check `sys.dm_db_missing_index_group_stats` for how often and how much benefit, consolidate overlapping suggestions into one well-designed index, and test.
- The DMV resets on restart and some changes; do not create indexes blindly from it.

```
SELECT TOP 20 ROUND(gs.avg_total_user_cost * gs.avg_user_impact * (gs.user_seeks + gs.user_scans), 0)
    AS score,
       d.statement AS table_name, d.equality_columns, d.inequality_columns, d.included_columns,
           gs.user_seeks, gs.avg_user_impact
FROM sys.dm_db_missing_index_group_stats gs
JOIN sys.dm_db_missing_index_groups g ON g.index_group_handle = gs.group_handle
JOIN sys.dm_db_missing_index_details d ON d.index_handle = g.index_handle
ORDER BY score DESC;
```

7.5 Fragmentation and maintenance: what matters

- Logical fragmentation hurts mostly **range scans from disk**. On SSD/NVMe and with data mostly in memory the effect is small.
- **Page density** (how full pages are) matters for memory and I/O efficiency. Low density wastes buffer pool.
- A common guideline: leave indexes under about 5% alone, reorganize between 5% and 30%, rebuild above 30%, and ignore small indexes (under roughly 1,000 pages). Treat it as a starting point and measure.
- **Statistics freshness usually matters more than fragmentation** for plan quality. Do not rebuild everything nightly out of habit; it generates log, blocks and invalidates plans.
- Use **ONLINE** and **RESUMABLE** options (edition permitting) for large rebuilds, and watch LCK_M_SCH_M waits.

```
SELECT OBJECT_NAME(ips.object_id) AS tbl, i.name AS index_name, ips.avg_fragmentation_in_percent,
    ips.page_count, ips.avg_page_space_used_in_percent
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'SAMPLED') ips
JOIN sys.indexes i ON i.object_id = ips.object_id AND i.index_id = ips.index_id
WHERE ips.page_count > 1000 AND ips.avg_fragmentation_in_percent > 5 ORDER BY
    ips.avg_fragmentation_in_percent DESC;
```

7.6 Special index types

Type	Use it for	Watch out
Clustered columnstore / nonclustered columnstore	Analytics: large scans and aggregations (batch mode, compression)	Not for singleton lookups; maintain rowgroup quality; delta store
Filtered index	Queries that always target a subset	Parameterised queries may not match; use OPTION (RECOMPILE) or literals
Indexed view	Pre-aggregated or joined results	Write overhead; strict requirements; use NOEXPAND on non-Enterprise editions
Computed column + index	Index on an expression (for example a derived date)	Must be deterministic; changes to the expression need a rebuild
Full-text index	Word search instead of LIKE '%word%'	Different query syntax and maintenance

8. Parameter Sniffing and Plan Changes

Two of the most common causes of 'sometimes slow' and 'suddenly slow' are **parameter sniffing** and **plan regressions**. Both come from the fact that SQL Server caches and reuses plans.

8.1 What parameter sniffing is

When a query or procedure with parameters is compiled, SQL Server looks at the **actual parameter values** of that first execution (it 'sniffs' them) and builds a plan suited to them. The plan is cached and **reused** for later calls, whatever the new values. This is a good thing: it saves compile CPU. It becomes a problem when data is skewed, because the best plan for 12 rows is a terrible plan for 2 million rows.

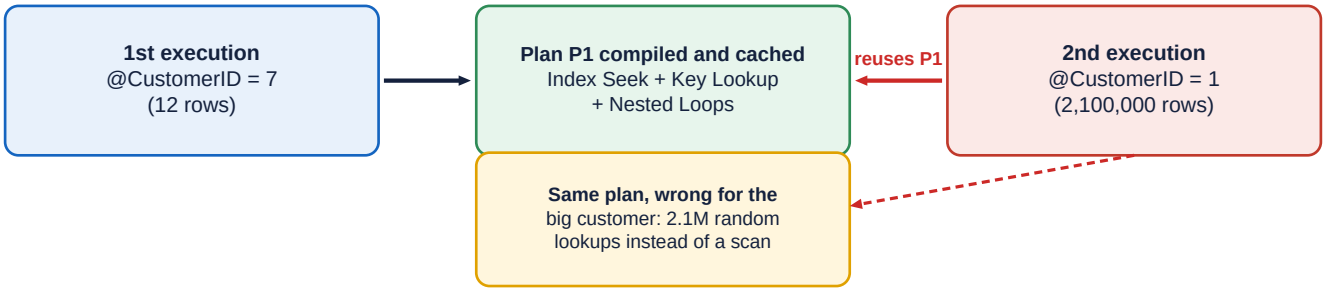


Figure 8.1 - A plan compiled for a small value is reused for a large value

8.2 How to recognise it

- The same query is fast for some parameter values and slow for others, or it is fast until the plan is evicted or recompiled and then slow.
- In the plan, **Parameter Compiled Value** differs from **Parameter Runtime Value**; estimated rows reflect the compiled value, actual rows reflect the runtime value.
- Query Store shows **several plans** for one query with very different durations, or one plan with high variation (max much larger than average).
- A restart, a statistics update or `sp_recompile` 'fixes' it for a while.

```
-- Query Store: queries with more than one plan and how each performed
SELECT q.query_id, p.plan_id, SUM(rs.count_executions) AS executions,
       SUM(rs.avg_duration * rs.count_executions) / NULLIF(SUM(rs.count_executions), 0) / 1000.0 AS
           avg_ms,
       MAX(rs.max_duration) / 1000.0 AS max_ms, MAX(rs.last_execution_time) AS last_run
FROM sys.query_store_query q
JOIN sys.query_store_plan p ON p.query_id = q.query_id
JOIN sys.query_store_runtime_stats rs ON rs.plan_id = p.plan_id
WHERE q.query_id IN (SELECT query_id FROM sys.query_store_plan GROUP BY query_id HAVING COUNT(*) > 1)
GROUP BY q.query_id, p.plan_id ORDER BY q.query_id, avg_ms DESC;
```

8.3 Options to fix it

Option	How	Trade-off
OPTION (RECOMPILE)	Compile for every run with the actual values	Correct plan each time; compile CPU on every execution; best for infrequent or highly variable queries
OPTIMIZE FOR (@p = value) / UNKNOWN	Build the plan for a typical value or an average	Stable but not ideal for all values
Parameter Sensitive Plan optimization (PSP)	Compatibility 160+ (SQL Server 2022): up to three variants chosen at run time	Automatic; look for Dispatcher/QueryVariantID in plans
Optional Parameter Plan Optimization (OPPO)	Compatibility 170 (SQL Server 2025): separate plans for NULL vs NOT NULL optional parameters	Helps 'WHERE (@p IS NULL OR col = @p)'
Query Store hints (2022+)	Attach RECOMPILE, MAXDOP, etc. to a query without code change	Needs Query Store; track hints
Plan forcing in Query Store	Force a known good plan	Forcing can fail after schema changes; monitor
Split the code path	Separate procedures or dynamic SQL for common vs rare cases	More code, most robust
Covering index	Remove lookups so both plans cost less	Does not remove skew but reduces the penalty
Do not: local-variable copies of parameters, DBCC FREEPROCCACHE in production	-	They hide values or hurt everyone

```
-- Per-query hint without changing the application (SQL Server 2022+ / Azure SQL with Query Store)
EXEC sys.sp_query_store_set_hints @query_id = 12345, @query_hints = N'OPTION (RECOMPILE)';
```

8.4 Plan regressions after statistics or version changes



Figure 8.2 - A plan regression found and fixed with Query Store plan forcing

A plan can change when statistics update, an index changes, the plan is evicted, the compatibility level is raised or the server restarts. If the new plan is worse, the query regresses at that moment. **Query Store** makes this visible: the time of the change, the old and new plan and the performance of each.

Step	Action
1	Open Query Store 'Regressed Queries' (or the Top Resource Consuming Queries report) for the affected time range
2	Open the query: compare plans and per-plan average duration, CPU and reads

Step	Action
3	Identify why the new plan is worse (estimates, join type, missing index, parallelism)
4	Short term: force the better plan (or let automatic plan correction do so)
5	Long term: fix the cause (statistics, index, query) and then unforce the plan
6	Monitor that forced plans stay valid (forcing failures appear in Query Store)

```
EXEC sys.sp_query_store_force_plan   @query_id = 12345, @plan_id = 6789;
EXEC sys.sp_query_store_unforce_plan @query_id = 12345, @plan_id = 6789;

-- Automatic plan correction: force the last good plan after a regression (SQL Server 2017+)
ALTER DATABASE [SalesDB] SET AUTOMATIC_TUNING (FORCE_LAST_GOOD_PLAN = ON);
SELECT reason, score, details FROM sys.dm_db_tuning_recommendations;
```

9. Query Patterns That Cause Slowness

Many slow queries share a small set of patterns. Learn to spot them in the plan and in the code.

9.1 Non-SARGable predicates

Why it is slow	A function or expression on the column prevents a seek.
Typical shape	`WHERE YEAR(OrderDate) = 2026 | WHERE LEFT(Name,3) = 'Abc' | WHERE ISNULL(Status,0) = 1 | WHERE Name LIKE '%son'`
Fix	Rewrite so the column is bare: date ranges, LIKE 'Abc%', explicit NULL handling; computed column with index for unavoidable expressions; full-text for contains search.

9.2 Implicit conversions

Why it is slow	A varchar column compared to an nvarchar parameter (common with ORMs) forces the *column* to be converted, so no seek and a bad estimate.
Typical shape	`WHERE AccountNumber = @p -- column varchar(20), @p nvarchar(4000)`
Fix	Make parameter types match column types (varchar parameters in the application); fix column types in the schema; find affected plans by searching for PlanAffectingConvert.

9.3 Scalar user-defined functions

Why it is slow	A scalar function called per row runs row by row, often forces a serial plan and hides its cost in a Compute Scalar.
Typical shape	`SELECT dbo.fn_GetStatusName(StatusID), ... FROM dbo.Orders`
Fix	Inline the logic, use a join to a lookup table, convert to an inline table-valued function, or rely on scalar UDF inlining (SQL Server 2019+, compatibility 150+) where eligible.

9.4 Table variables and multi-statement table functions

Why it is slow	No statistics (1-row or fixed estimates) lead to nested loops and tiny grants on big data.
Typical shape	`DECLARE @t TABLE (...); INSERT ... 500000 rows; JOIN @t ...`
Fix	Use #temp tables (statistics, indexes); inline TVFs; compatibility 150+ for deferred compilation.

9.5 OR and optional-parameter filters

Why it is slow	One plan has to serve many combinations.
Typical shape	`WHERE (@Name IS NULL OR Name = @Name) AND (@City IS NULL OR City = @City)`
Fix	Parameterised dynamic SQL that adds only the active filters; OPTION (RECOMPILE); OPPO in SQL Server 2025.

9.6 NOT IN and correlated subqueries

Why it is slow	NOT IN with a nullable subquery is slow and can return wrong results; correlated subqueries can become per-row loops with spools.
Typical shape	`WHERE CustomerID NOT IN (SELECT CustomerID FROM dbo.Orders)`
Fix	Use NOT EXISTS; join to a pre-aggregated derived table; add the index the spool is simulating.

9.7 DISTINCT or GROUP BY to hide duplicates

Why it is slow	A join multiplies rows and DISTINCT cleans up with a big sort or hash.
Typical shape	`SELECT DISTINCT c.* FROM Customers c JOIN Orders o ON ...`
Fix	Use EXISTS (semi join) or aggregate the many-side first.

9.8 SELECT * and wide result sets

Why it is slow	Prevents covering indexes, widens sorts and hashes (more memory and spills), increases network time.
Typical shape	`SELECT * FROM dbo.Orders o JOIN ...`
Fix	Select only needed columns; add paging; return aggregates not rows.

9.9 Row-by-row processing (cursors, loops)

Why it is slow	Thousands of tiny statements instead of one set-based statement.
Typical shape	`WHILE ... FETCH NEXT ... UPDATE ...`
Fix	Rewrite set-based; batch DML; use window functions or APPLY.

9.10 Paging with large offsets

Why it is slow	OFFSET/FETCH reads and discards all skipped rows.
Typical shape	`ORDER BY OrderID OFFSET 500000 ROWS FETCH NEXT 50 ROWS ONLY`
Fix	Keyset paging: WHERE OrderID > @LastSeen ORDER BY OrderID with TOP (50).

9.11 Window functions with the default frame

Why it is slow	The default RANGE frame uses an on-disk Window Spool.
Typical shape	`SUM(Amount) OVER (ORDER BY OrderDate)`
Fix	Specify ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW; consider batch mode.

9.12 Very long IN lists

Why it is slow	Huge constant lists lead to compile cost and poor estimates.
Typical shape	`WHERE ID IN (1,2,3,... 5000 values ...)`
Fix	Load values into a temp table or table-valued parameter and join.

9.13 Large UPDATE/DELETE statements

Why it is slow	Wide plans, many locks, log growth, blocking.
Typical shape	`DELETE FROM dbo.AuditLog WHERE CreatedAt < '2026-01-01'`
Fix	Delete in batches (for example 5,000-50,000 rows), short transactions, indexes on the filter column, drop unused indexes.

9.14 ORM and application patterns

Why it is slow	N+1 queries, nvarchar parameters, SELECT of whole entities, no paging, long-lived transactions.
Typical shape	`1 query for orders + 1 query per order for lines`
Fix	Eager loading or joins; configure parameter types; select projections; set command timeouts; keep transactions short.

9.15 Deeply nested views and functions

Why it is slow	The optimizer must expand and combine many layers; estimates degrade; compile time grows.
Typical shape	`view of view of view joined to tables`
Fix	Flatten; materialise intermediate results in temp tables; use indexed views where suitable.

10. Wait Statistics for Troubleshooting

When a query is not on the CPU it is **waiting**, and SQL Server records why. Wait statistics tell you which resource is the bottleneck, which narrows the search from 'everything' to one area.

10.1 How to use waits

- A worker thread is **running** (using CPU), **runnable** (ready, waiting for a CPU) or **suspended** (waiting for a resource). **Wait time = resource wait + signal wait** (time spent runnable).
- Use **deltas** over the slow period, not totals since startup. Remove idle and background waits (SLEEP_TASK, LAZYWRITER_SLEEP, XE_TIMER_EVENT and similar).
- **Signal wait above about 20-25%** of total wait time suggests CPU pressure.
- A wait names a **resource**, not a cause: PAGEIOLATCH means 'reading from disk', not whether storage is slow or the query reads too much.
- Look at waits at three levels: the whole instance (sys.dm_os_wait_stats), a session or query (sys.dm_exec_session_wait_stats, plan WaitStats, Query Store) and right now (sys.dm_os_waiting_tasks).

```
-- Waits during a 60-second window, benign waits removed
SELECT wait_type, waiting_tasks_count, wait_time_ms, signal_wait_time_ms INTO #w1 FROM
    sys.dm_os_wait_stats;
WAITFOR DELAY '00:01:00';
SELECT TOP 15 a.wait_type, a.waiting_tasks_count - b.waiting_tasks_count AS waits,
       a.wait_time_ms - b.wait_time_ms AS wait_ms, a.signal_wait_time_ms - b.signal_wait_time_ms AS
           signal_ms,
       (a.wait_time_ms - b.wait_time_ms) * 1.0 / NULLIF(a.waiting_tasks_count - b.waiting_tasks_count,
           0) AS avg_ms
FROM sys.dm_os_wait_stats a JOIN #w1 b ON b.wait_type = a.wait_type
WHERE a.wait_time_ms > b.wait_time_ms
  AND a.wait_type NOT IN
      ('SLEEP_TASK','LAZYWRITER_SLEEP','CHECKPOINT_QUEUE','XE_TIMER_EVENT','XE_DISPATCHER_WAIT','REQUEST
      _FOR_DEADLOCK_SEARCH',
      'SQLTRACE_INCREMENTAL_FLUSH_SLEEP','BROKER_TO_FLUSH','BROKER_EVENTHANDLER','DIRTY_PAGE_POLL','LOGM
          GR_QUEUE','ONDEMAND_TASK_QUEUE',
      'WAITFOR','SP_SERVER_DIAGNOSTICS_SLEEP','QDS_PERSIST_TASK_MAIN_LOOP_SLEEP','QDS_ASYNC_QUEUE','FT_I
          FTS_SCHEDULER_IDLE_WAIT',
      'SLEEP_SYSTEMTASK','BROKER_TASK_STOP','BROKER_TRANSMITTER','SQLTRACE_BUFFER_FLUSH','CLR_AUTO_EVENT
          ','CLR_MANUAL_EVENT',
      'DISPATCHER_QUEUE_SEMAPHORE','HADR_WORK_QUEUE','HADR_TIMER_TASK','HADR_NOTIFICATION_DEQUEUE','HADR
          _LOGCAPTURE_WAIT','REDO_THREAD_PENDING_WORK')
ORDER BY wait_ms DESC;
DROP TABLE #w1;
```

10.2 The waits you will meet most often

Wait type	Means	Usual causes	First actions
SOS_SCHEDULER_YIELD	Thread used its CPU time slice and went back to the runnable queue	CPU-bound queries, too few CPUs	Find top CPU queries; tune plans; check signal waits and CPU
THREADPOOL	No free worker thread	Blocking pile-ups, too much parallelism, connection storms	Fix head blocker; reduce DOP; use pooling
CXPACKET / CXCONSUMER / CXSYNC_PORT	Parallel threads waiting for each other at an exchange	Skew, one slow thread, DOP too high	Find the slow thread's other wait; fix estimates; tune MAXDOP and cost threshold
PAGEIOLATCH_SH (and _EX, _UP)	Waiting to read a data page from disk	Slow storage, large scans, too little memory	Reduce reads (indexes, plans); check file latency; add memory
IO_COMPLETION	Non-data synchronous I/O, such as spills to tempdb	Sort/hash spills, slow tempdb	Fix spills (estimates, grants); speed tempdb
WRITELOG	Waiting for the transaction log flush at commit	Slow log storage, many small commits, AG synchronous commit	Faster log disk; batch commits; check HADR_SYNC_COMMIT
LOGBUFFER / LOGMGR_RESERVE_APPEND	Log buffer full / log space needed	Log device too slow; log full or cannot grow	Faster log; check log_reuse_wait_desc
LCK_M_S / LCK_M_U / LCK_M_X / LCK_M_IX	Waiting for a lock held by another session	Long transactions, missing indexes, readers vs writers	Find head blocker; RCSI; indexes; shorter transactions
LCK_M_SCH_M / LCK_M_SCH_S	Schema modification vs stability lock	DDL or index rebuild during busy time	Run DDL off-peak; use WAIT_AT_LOW_PRIORITY
PAGELATCH_EX / _UP / _SH	In-memory page latch contention	tempdb allocation, last-page inserts	tempdb files; OPTIMIZE_FOR_SEQUENTIAL_KEY; key design
LATCH_EX / LATCH_SH	Non-buffer latch contention	Specific internal structure (see sys.dm_os_latch_stats)	Identify the latch class; reduce the cause
RESOURCE_SEMAPHORE	Waiting for a query memory grant	Over-estimated grants, many big queries, low memory	Fix estimates; grant hints; schedule reports; add memory
RESOURCE_SEMAPHORE_QUERY_COMPILE	Compilation throttled	Many ad hoc compiles	Parameterise; optimize for ad hoc
CMEMTHREAD	Contention on a thread-safe memory object	Heavy ad hoc workload on many cores	Parameterise; update builds
ASYNC_NETWORK_IO	Server waiting for the client to read results	Slow client, large results, network	Return fewer rows; fix client code
HADR_SYNC_COMMIT	Commit waiting for synchronous AG secondary	Network or secondary log latency	Improve secondary I/O/network; consider async
OLEDB	Linked server or OLE DB call	Slow remote server, large remote results	Filter remotely; copy data locally
PREEMPTIVE_OS_*	Thread in an OS call outside the scheduler	Authentication, file operations, CRL checks	Check the matching external system (DC, storage, antivirus)
BACKUPIO / BACKUPTHREAD	Backup activity	Backup running	Normal; tune backup throughput

Part of the earlier companion guide on execution plans and wait types goes deeper on every wait; this chapter is the quick working list for a live incident.

10.3 Waits for one slow query

```
-- Waits of one session since it connected
SELECT wait_type, waiting_tasks_count, wait_time_ms, signal_wait_time_ms
FROM sys.dm_exec_session_wait_stats WHERE session_id = 57 ORDER BY wait_time_ms DESC;

-- Query Store: which wait categories hurt a given query
SELECT ws.wait_category_desc, SUM(ws.total_query_wait_time_ms) AS total_wait_ms
FROM sys.query_store_wait_stats ws JOIN sys.query_store_plan p ON p.plan_id = ws.plan_id
WHERE p.query_id = 12345 GROUP BY ws.wait_category_desc ORDER BY total_wait_ms DESC;
```

11. Blocking and Deadlocks

Blocking means one session waits for a lock held by another. **Deadlock** means two sessions each wait for the other, and SQL Server kills one of them (error 1205). Blocking is the most common reason a fast query is sometimes slow.

11.1 Finding the head blocker

```
-- Who is blocked, by whom, on what, for how long
SELECT r.session_id AS blocked, r.blocking_session_id AS blocker, r.wait_type, r.wait_time / 1000.0 AS
    wait_s, r.wait_resource,
       DB_NAME(r.database_id) AS db, SUBSTRING(t.text, 1, 200) AS blocked_sql
FROM sys.dm_exec_requests r CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.blocking_session_id <> 0 ORDER BY r.wait_time DESC;

-- Head blockers: block others but are not blocked themselves, with their last SQL and open transaction
    age
SELECT s.session_id, s.login_name, s.host_name, s.program_name, s.status, s.last_request_start_time,
       tat.transaction_begin_time, DATEDIFF(MINUTE, tat.transaction_begin_time, GETDATE()) AS
           tran_minutes, t.text AS last_sql
FROM sys.dm_exec_sessions s
LEFT JOIN sys.dm_tran_session_transactions st ON st.session_id = s.session_id
LEFT JOIN sys.dm_tran_active_transactions tat ON tat.transaction_id = st.transaction_id
LEFT JOIN sys.dm_exec_connections c ON c.session_id = s.session_id
OUTER APPLY sys.dm_exec_sql_text(c.most_recent_sql_handle) t
WHERE s.session_id IN (SELECT DISTINCT blocking_session_id FROM sys.dm_exec_requests WHERE
    blocking_session_id <> 0)
  AND s.session_id NOT IN (SELECT session_id FROM sys.dm_exec_requests WHERE blocking_session_id <> 0);
```

A head blocker in **sleeping** status with an open transaction is the classic 'forgotten transaction': the application started a transaction and is not committing. A head blocker that is **running** is doing long work while holding locks.

11.2 Common causes and fixes

Cause	Evidence	Fix
Long or abandoned transactions	Head blocker sleeping with an old transaction_begin_time	Fix the application (commit/rollback, no user interaction inside transactions, timeouts); set XACT_ABORT
Updates/deletes scanning many rows	Update plans with scans; many locks	Index the WHERE columns; batch the work
Readers blocked by writers	LCK_M_S waits on read queries	READ_COMMITTED_SNAPSHOT or SNAPSHOT isolation; correct indexing
Lock escalation to table level	Table lock (LCK_M_IX/S/X at OBJECT level) after about 5,000 locks	Batch modifications; indexes; consider partitioning; escalation settings as a last resort
DDL or index rebuild at peak	LCK_M_SCH_M blocking everything behind it	Run off-peak; online operations with WAIT_AT_LOW_PRIORITY
Serializable / HOLDLOCK ranges	Range locks (LCK_M_RS_*, RIn_*)	Use a lower isolation level where phantoms are acceptable
Hot rows or hot pages (counters, queue tables)	Many sessions waiting on the same key	Redesign the hot spot: sequences, partitioned counters, READPAST/UPDLOCK queue pattern
Blocking by backups, DBCC or maintenance	System sessions as blockers	Schedule outside business hours; throttle

11.3 Optimistic isolation: RCSI and snapshot

- **READ_COMMITTED_SNAPSHOT (RCSI)** makes readers read the last committed version instead of waiting for locks. It removes most reader/writer blocking with no application change, at the cost of a version store in tempdb (or the persistent version store with accelerated database recovery) and more tempdb I/O.
- **SNAPSHOT isolation** gives a consistent view for the whole transaction and can cause update conflicts (error 3960).
- Test the workload: code that relies on blocking for correctness (for example reading then updating based on the read) may need UPDLOCK hints.

```
ALTER DATABASE [SalesDB] SET READ_COMMITTED_SNAPSHOT ON WITH ROLLBACK IMMEDIATE;   -- needs exclusive
    access briefly; plan a window
SELECT name, is_read_committed_snapshot_on, snapshot_isolation_state_desc FROM sys.databases;
```

11.4 Deadlocks

A deadlock graph is captured automatically by the **system_health** Extended Events session. Read it in this order: the **victim**, then each **process** (statement, isolation level, locks held and wanted), then the **resource list** (which objects/keys), and the **owner/waiter** arrows that form the cycle.

```
-- Recent deadlock graphs from the system_health session (ring buffer)
SELECT XEvent.value('(event/@timestamp)[1]', 'datetime2') AS event_time,
       XEvent.query('(event/data/value/deadlock)[1]') AS deadlock_graph
FROM (SELECT CAST(st.target_data AS xml) AS target_data
      FROM sys.dm_xe_session_targets st JOIN sys.dm_xe_sessions s ON s.address =
          st.event_session_address
      WHERE s.name = 'system_health' AND st.target_name = 'ring_buffer') d
CROSS APPLY d.target_data.nodes('RingBufferTarget/event[@name="xml_deadlock_report"]') AS x(XEvent)
ORDER BY event_time DESC;
```

Deadlock pattern	Fix
Two transactions update the same two tables in opposite order	Access objects in the same order everywhere
Reader and writer deadlock (key lookup vs update on the same row)	Covering index to remove the lookup, or RCSI
Conversion deadlock (S then X on the same resource)	Use UPDLOCK when reading rows you will update
Long transactions touching many rows	Shorter transactions, smaller batches, indexes
Foreign key checks and cascades	Index foreign key columns; review cascades
Application	Retry the transaction on error 1205 (with a short random delay)

11.5 Handling an incident

- Identify the head blocker and its owner. If the transaction is abandoned, the owner may be able to commit or roll back.
- **KILL** as a last resort: rolling back a large transaction can take as long as it ran and may block even longer (check progress with KILL ... WITH STATUSONLY).
- Afterwards, find the cause in code or design; do not rely on kills.
- Set sensible **lock timeouts and command timeouts** in the application so that sessions do not wait forever.

12. Resource Bottlenecks

When waits and counters point at a resource, work through the matching section: what you see, what to check and how to fix. Always ask first whether queries are doing **too much work**; adding hardware is a last step.

12.1 CPU

Evidence	Check	Fix
CPU near 100%; many runnable tasks; SOS_SCHEDULER_YIELD; high signal wait share	Top CPU queries (Query Store, dm_exec_query_stats); compilation rate; parallelism	Tune top queries; remove scalar UDFs; fix scans and implicit conversions; reduce unnecessary parallelism; reduce compiles; add CPU last
High CPU only at certain times	Scheduled jobs, reports, statistics updates, index maintenance	Reschedule; throttle; resource governor
High CPU after upgrade or failover	Plan changes, compatibility level, cold cache, missing statistics	Compare plans in Query Store; update statistics; force plans
High CPU with many compilations	SQL Compilations/sec vs Batch Requests/sec	Parameterise; optimize for ad hoc; plan cache (chapter 14)

12.2 Memory, grants and spills

Sorts and hash operations get a **memory grant** before the query starts, sized from *estimated* rows. Too small means **spills** to tempdb; too large means other queries wait (RESOURCE_SEMAPHORE).

Evidence	Check	Fix
Sort/Hash warnings in plans; IO_COMPLETION; heavy tempdb I/O	Estimates vs actuals at the sort/hash; row width	Fix estimates (statistics, sniffing, table variables); reduce columns carried; index for order; feedback features
RESOURCE_SEMAPHORE waits; grants pending	sys.dm_exec_query_memory_grants (granted NULL)	Fix over-estimates; MAX_GRANT_PERCENT hint; Resource Governor; schedule reports; add memory
Low and unstable page life expectancy; high page reads	Memory settings (max server memory); working set vs RAM; large scans	Fewer reads (indexes); right-size max server memory; add memory
Memory pressure in the OS	Windows paging, other processes, max server memory too high	Leave memory for the OS; check lock pages in memory policy

```
SELECT session_id, dop, requested_memory_kb, granted_memory_kb, used_memory_kb, max_used_memory_kb,
    ideal_memory_kb, wait_time_ms
FROM sys.dm_exec_query_memory_grants ORDER BY requested_memory_kb DESC;
```

12.3 Disk I/O and storage

Evidence	Check	Fix
PAGEIOLATCH_SH high; read latency above 10-20 ms (data)	sys.dm_io_virtual_file_stats; Perfmon sec/Read; queries with most physical reads	Reduce reads (indexes, plan fixes); add memory; faster or less shared storage; correct multipath and queue depth
Write latency high; checkpoint spikes	Write stalls on data files; storage cache	Spread files; faster storage; check backups overlapping peak
Slow only after restart/failover	Cold cache; physical reads	Warm-up queries; accept; faster storage
Autogrowth events	Default trace / error log; file growth increments	Pre-size files; sensible growth sizes; instant file initialisation for data files
Antivirus or backup agents scanning data files	Process activity on the host	Exclusions per Microsoft guidance

12.4 tempdb

Evidence	Check	Fix
PAGELATCH_UP/EX on pages 2:1:1, 2:1:2, 2:1:3 (allocation pages)	sys.dm_os_waiting_tasks resource_description; number of tempdb data files	Several equally sized tempdb files (commonly up to 8 or the core count); cache temp objects; memory-optimized tempdb metadata (2019+)
tempdb full or growing fast	Space by session and by version store	Find the consumer (below); fix spills and huge temp tables; limit version store growth by ending long transactions
Slow tempdb I/O	File latency for tempdb files	Faster/local NVMe for tempdb; separate from busy volumes

```
-- Who is using tempdb space right now (user objects, internal objects such as spills and sorts)
SELECT TOP 10 s.session_id, s.login_name, s.program_name,
       (u.user_objects_alloc_page_count - u.user_objects_dealloc_page_count) * 8 / 1024 AS user_obj_mb,
       (u.internal_objects_alloc_page_count - u.internal_objects_dealloc_page_count) * 8 / 1024 AS
           internal_obj_mb
FROM sys.dm_db_session_space_usage u JOIN sys.dm_exec_sessions s ON s.session_id = u.session_id
ORDER BY u.internal_objects_alloc_page_count + u.user_objects_alloc_page_count DESC;
-- Version store size
SELECT SUM(version_store_reserved_page_count) * 8 / 1024 AS version_store_mb FROM
    tempdb.sys.dm_db_file_space_usage;
```

12.5 Transaction log

Evidence	Check	Fix
WRITELOG / LOGBUFFER waits; log write latency above a few ms	sys.dm_io_virtual_file_stats on the log; commits per second	Faster log storage; larger transactions per commit; reduce logging; delayed durability only if acceptable
Log full, error 9002, LOGMGR_RESERVE_APPEND	log_reuse_wait_desc; open transactions; log backup chain	Take log backups; fix the reuse blocker (replication, AG, long transaction); add space
Log file grows huge after one job	Large single transactions	Batch the work; use minimal logging patterns where valid; right-size the log
Slow recovery or failover	Very large number of VLFs; long open transactions	Right-size growth; avoid tiny autogrowths

```
SELECT name, recovery_model_desc, log_reuse_wait_desc FROM sys.databases;
SELECT * FROM sys.dm_db_log_space_usage;     -- in the database
```

12.6 Parallelism and worker threads

Evidence	Check	Fix
CXPACKET high with slow queries; one thread with most rows	Actual Rows per Thread; the slow thread's other wait	Fix estimates/skew; adjust MAXDOP and cost threshold; indexes
THREADPOOL waits; connections time out	Blocking pile-up; sys.dm_os_schedulers work_queue_count	Resolve head blocker; reduce DOP; use pooling; raise cost threshold
Parallel plan slower than serial	Compare OPTION (MAXDOP 1) timings	Fix the plan; lower DOP; DOP feedback (2022+)
Serial plan where parallel expected	Non Parallel Plan Reason in the plan	Remove inhibitors (scalar UDFs, table variable DML)

Check the settings once

Cost threshold for parallelism (default 5 is usually too low), MAXDOP (often 8 or per NUMA node), max server memory, tempdb file layout, optimize for ad hoc workloads and compatibility level. Misconfigured defaults cause many 'mystery' slowdowns.

13. Client, Network and Application Causes

Sometimes the database finishes quickly and the user still waits. The time is spent sending results, waiting on the client, or in the application between calls.

13.1 ASYNC_NETWORK_IO

SQL Server has results ready but is waiting for the client to **acknowledge and consume** them. It is the single most common 'slow query' that is not a database problem.

Cause	How to confirm	Fix
Application reads rows slowly (processing each row, writing to a file or UI while reading)	Session in ASYNC_NETWORK_IO; high rows returned; client CPU or thread busy	Read results into memory first or process efficiently; fast data readers
Very large result sets	Rows and bytes returned; statement is SELECT * with no filter	Filter and aggregate on the server; paging; avoid SELECT *
Client machine overloaded (CPU, memory)	Client performance counters	Fix the client; move to a stronger host
Network problems	Latency/packet loss; drivers, firewalls, VPN, wrong MTU	Network team; check adapter drivers and settings
SSMS grid output for huge results	Query is fast with 'Discard results after execution'	Test with discard results; do not judge speed by grid rendering

13.2 Application patterns

Pattern	Symptom	Fix
N+1 queries (one query per row of a parent set)	Thousands of identical small queries; high execution_count; each is fast	Join or batch; eager loading; cache lookups
Connection pool exhaustion or no pooling	Many connections, login storms, timeouts; THREADPOOL; slow logins	Enable pooling; close connections promptly; right-size pool
Long-lived transactions in the application	Blocking; sleeping sessions with open transactions	Short transactions; commit before user interaction
Missing or huge command timeouts	Errors at 30 s or hangs for minutes	Set explicit timeouts; handle retries
Parameter type mismatch (nvarchar for varchar)	Implicit conversion warnings; scans	Specify exact parameter types and lengths
Chatty round trips over WAN	Fast statements, slow page	Batch calls; stored procedures; table-valued parameters; locate app near the database
MARS (multiple active result sets) misuse	MSQL_* waits, odd blocking	Avoid MARS unless needed
Retries without backoff	Storms after a blip	Exponential backoff

```
-- Connections and request counts by application and host (spot a runaway client or pool leak)
SELECT s.program_name, s.host_name, COUNT(*) AS sessions, SUM(CASE WHEN r.session_id IS NOT NULL THEN 1
    ELSE 0 END) AS running,
       SUM(CASE WHEN s.status = 'sleeping' AND EXISTS (SELECT 1 FROM sys.dm_tran_session_transactions t
           WHERE t.session_id = s.session_id) THEN 1 ELSE 0 END) AS sleeping_with_tran
FROM sys.dm_exec_sessions s LEFT JOIN sys.dm_exec_requests r ON r.session_id = s.session_id
WHERE s.is_user_process = 1 GROUP BY s.program_name, s.host_name ORDER BY sessions DESC;
```

13.3 Linked servers and distributed queries

- Waits: `OLEDB`, `MSQL_DQ`, `ASYNC_NETWORK_IO`. Plans show Remote Query / Remote Scan operators.
- The optimizer has poor statistics for remote tables and often pulls the whole table. **Push filters to the remote server** with OPENQUERY or by running the query there; copy needed data locally (temp table) and join locally.
- Distributed transactions (MSDTC) add overhead; avoid them when possible.

13.4 Network and connectivity checks

- **Compare timings from the app server and from the database server** with the same query (sqlcmd). If local is fast and remote slow, look at the network, driver or client.
- Check **DNS** and name resolution delays at connect, **TLS/encryption** overhead, and slow **authentication** (PREEMPTIVE_OS_AUTHENTICATIONOPS, domain controller latency).
- Look at **packet size** and drivers; very old drivers can be slow with large results.

14. Plan Cache and Compilation Problems

Compiling a plan costs CPU and memory. Healthy systems compile a query once and reuse the plan thousands of times. Unhealthy ones compile constantly.

14.1 Signs of compile trouble

- **SQL Compilations/sec** or **Re-Compilations/sec** are a large share of **Batch Requests/sec**.
- High CPU with RESOURCE_SEMAPHORE_QUERY_COMPILE or CMEMTHREAD waits.
- Plan cache full of **single-use plans** (ad hoc SQL with literals).
- Plans appear 'new' every time; the same query_hash has thousands of query_text variants.
- Compile time in the plan or STATISTICS TIME is a big part of total elapsed time.

```
-- How much of the plan cache is single-use ad hoc plans?
SELECT objtype, COUNT(*) AS plans, SUM(CASE WHEN usecounts = 1 THEN 1 ELSE 0 END) AS single_use,
       SUM(size_in_bytes) / 1024 / 1024 AS cache_mb,
       SUM(CASE WHEN usecounts = 1 THEN size_in_bytes ELSE 0 END) / 1024 / 1024 AS single_use_mb
FROM sys.dm_exec_cached_plans GROUP BY objtype ORDER BY cache_mb DESC;

-- Same query shape compiled many times (different literals)
SELECT TOP 20 qs.query_hash, COUNT(*) AS variants, SUM(qs.execution_count) AS executions, MIN(st.text)
    AS sample_text
FROM sys.dm_exec_query_stats qs CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
GROUP BY qs.query_hash HAVING COUNT(*) > 50 ORDER BY variants DESC;
```

14.2 Fixes for ad hoc plan bloat

Fix	How	Notes
Parameterise in the application	Use parameterised commands or sp_executesql	Best fix; also improves security
optimize for ad hoc workloads	sp_configure; stores a small stub on first use and the full plan on the second	Low risk; reduces memory waste
Forced parameterization	ALTER DATABASE ... SET PARAMETERIZATION FORCED	Test: it can introduce parameter sniffing issues; use plan guides/hints if needed
OPTIMIZED_SP_EXECUTESQL (SQL Server 2025)	Database-scoped option treating sp_executesql like procedures for compilation	Reduces compile storms
Avoid clearing the cache	Do not schedule DBCC FREEPROCCACHE	Clearing forces mass recompilation and cold plans

14.3 Recompilations

A **recompile** happens when a cached plan is no longer valid or a statement asks for a new one. Frequent recompiles of expensive statements burn CPU and can change plans.

Cause	Example	What to do
Statistics changed	Auto update after many row changes	Normal; ensure stats updates are not excessive
Schema change	ALTER TABLE, index created/dropped	Avoid in busy periods
Temp table changes inside a procedure	Creating and filling #temp, then querying	Create temp tables up front; avoid DDL after first use

Cause	Example	What to do
SET option changes inside a procedure	SET ANSI_NULLS and similar	Set at connection level, not inside code
OPTION (RECOMPILE) or WITH RECOMPILE	Used deliberately	Remove if not needed
Plan cache eviction by memory pressure	Cache trimmed	Fix memory pressure; reduce single-use plans

```
-- Capture why statements recompile (short session; low overhead)
CREATE EVENT SESSION [recompiles] ON SERVER
ADD EVENT sqlserver.sql_statement_recompile (ACTION (sqlserver.sql_text, sqlserver.database_name))
ADD TARGET package0.ring_buffer (SET max_memory = (4096))
WITH (MAX_DISPATCH_LATENCY = 5 SECONDS);
ALTER EVENT SESSION [recompiles] ON SERVER STATE = START;   -- event data includes recompile_cause
```

14.4 Dangerous 'fixes'

Action	Why it is dangerous
DBCC FREEPROCCACHE on production	Every query recompiles at once; CPU spike; possible new bad plans
DBCC DROPCLEANBUFFERS on production	Empties the data cache; every query reads from disk
Restarting SQL Server to 'clear' a problem	Loses cache, statistics of waits and evidence; problem returns
Blanket NOLOCK hints	Dirty reads, missing or duplicate rows; use RCSI instead
Forcing a join or index hint everywhere	Becomes wrong as data changes
Killing sessions without understanding	Long rollbacks; masks the cause

15. The Tuning Toolbox and Safe Change Process

Once you know the cause, pick the **smallest, safest** fix that removes it, test it, and keep a way back.

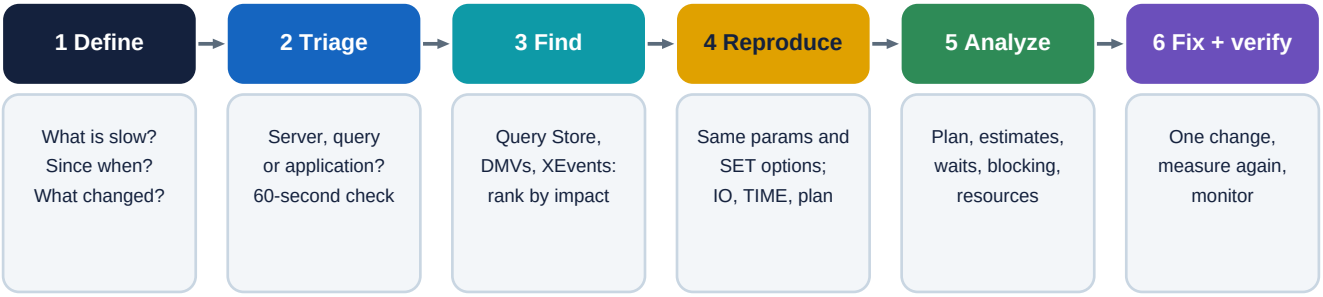


Figure 15.1 - The slow-query runbook

15.1 Choosing the fix

Tool	Use when	Risk	Notes
Update statistics	Estimates are stale or sampled poorly	Low (plans may change)	FULLSCAN for skewed critical tables; off-peak for very large tables
Add or change an index	Scans, lookups or sorts the plan cannot avoid	Medium (write cost, space, plan changes)	Create online where supported; test the impact on writes
Rewrite the query	Non-SARGable logic, bad patterns, implicit conversions	Low if tested	Best long-term fix; needs deployment
Fix data types / schema	Mismatched types, missing constraints, wrong keys	Medium	May need planned change
OPTION (RECOMPILE) / OPTIMIZE FOR	Parameter sniffing	Low to medium	Targeted, reversible
Query Store hints / plan forcing	Regression or sniffing when code cannot change	Medium (forcing can fail; hides drift)	Monitor and revisit
Table or query hints (INDEX, FORCESEEK, JOIN)	Last resort, with evidence	High over time	Document why; re-test after data changes
Compatibility level / CE settings	Planned upgrade or regression from the new estimator	Medium	Test with Query Store comparison
Server settings (MAXDOP, cost threshold, memory)	Misconfiguration	Medium (server-wide)	Change one at a time, with a rollback
Hardware / service tier	Healthy queries on saturated resources	Cost	After you removed waste

15.2 A safe change process

Step	Action
1 Baseline	Record duration, CPU, logical reads, memory grant, plan (save the .sqlplan), waits and rows returned for slow and typical parameters
2 Hypothesis	Write down the cause you believe and the change that should fix it
3 Test off-production	Use a recent restore or a test replica with production-sized data; compare before and after
4 One change at a time	Otherwise you cannot tell what helped, and you may cause new problems
5 Check side effects	Other queries on the same tables; write performance for new indexes; plan changes elsewhere (Query Store)
6 Plan deployment	Maintenance window if locking or recompiles are possible; online options; rollback script ready
7 Monitor after release	Watch the same metrics for hours or days; compare with the baseline
8 Document	Cause, change, result, rollback and who approved it

Metric	Before	After	Comment
Elapsed time (slow parameter)			
CPU time			
Logical reads (largest table)			
Memory grant / spills			
Estimated vs actual rows (key operator)			
Plan shape / hash			
Top wait			

Copy this table into your ticket and fill it in before and after the change.

15.3 Index changes in production

- Create with `WITH (ONLINE = ON)` where the edition allows it; otherwise use a window. Large builds can generate lots of log and tempdb activity.
- Check how many writes hit the table: each extra index adds write cost and memory use.
- To test dropping an index, **disable** it first (ALTER INDEX ... DISABLE) so it can be rebuilt quickly if queries suffer. Disabling a clustered index makes the table unreadable, so never do that.
- Query plans for affected queries will **recompile** when an index appears or disappears.

15.4 Using hints responsibly

- A hint overrides the optimizer's choice using today's data. Tomorrow's data may need a different plan.
- Prefer fixing statistics, indexes and code. If you must hint, choose the narrowest option (for example OPTION (RECOMPILE) or a Query Store hint) and add a comment with the reason, date and ticket.
- Review hints regularly: after data growth, upgrades and compatibility changes.

16. Special Scenarios

These situations come up so often that they deserve their own short playbooks.

16.1 Slow after an upgrade or compatibility level change

- Enable **Query Store before** the change to capture a baseline. Raise the compatibility level in a test copy first.
- After the change use the **Regressed Queries** report; force the old plan for regressions while you investigate.
- A regression can come from the new cardinality estimator (compatibility 120+), batch mode on rowstore (150+), parameter sensitive plans (160+) or other IQP changes. Use `USE HINT('FORCE_LEGACY_CARDINALITY_ESTIMATION')` or `QUERY_OPTIMIZER_COMPATIBILITY_LEVEL_n` per query as a stopgap.
- Update statistics after major version upgrades unless the upgrade process already did.

16.2 Slow after a restart or failover

- The buffer pool and plan cache are empty: the first runs read from disk and compile. Expect elevated PAGEIOLATCH waits and compile CPU for a while.
- Plans may differ from before. Check Query Store for plan changes; force known good plans for critical queries.
- Consider a warm-up routine (run key queries or reads) and Query Store hints for known problem queries.
- On an availability group, check that the new primary has current statistics (secondary statistics are separate unless synchronised) and that readable-secondary workloads moved appropriately.

16.3 Slow at certain times

When	Likely cause	Check
Fixed time each day or week	Scheduled job, report, ETL, backup, index/statistics maintenance, DBCC CHECKDB	SQL Agent job history; correlate with Query Store intervals
Month-end / peak business	Larger data volume or concurrency	Compare row counts and waits to a normal day
After log backup or checkpoint	I/O spikes	File latency; storage contention with backups
During autogrowth	File growth blocking writes (log always zero-initialised)	Default trace / events; pre-size files
When statistics update	Recompile storm and new plans	Recompile events; plan changes in Query Store

16.4 Slow INSERT, UPDATE and DELETE

Cause	Evidence	Fix
Too many indexes	Wide update plan; many Index Insert/Update operators	Drop unused indexes (verify first); consolidate
Log latency	WRITELOG waits	Faster log; batch commits
Triggers	Inserted/Deleted Scan; hidden cost in plans	Optimise or remove trigger logic; set-based triggers
Foreign key checks without indexes	Assert and scans on child tables	Index foreign key columns
Blocking	LCK_M_X waits	Shorter transactions; indexes; RCSI for readers
Page splits and fragmentation on random keys	High page splits/sec	Fill factor; sequential keys; OPTIMIZE_FOR_SEQUENTIAL_KEY for hot last page
Large single statements	Huge log and long locks	Batch in chunks

16.5 Slow reports and analytics

- Large scans and aggregations are best served by **columnstore indexes** and **batch mode** (columnstore, or batch mode on rowstore at compatibility 150+).
- Use **indexed views** or summary tables for repeated aggregates; schedule heavy reports apart from OLTP peaks, or read from a replica.
- Watch for **spills** (large sorts and hashes), RESOURCE_SEMAPHORE queues and parallel plan skew.
- Check statistics on the large fact tables and that predicates allow partition and segment elimination.

16.6 Slow stored procedures: find the slow statement

A procedure is only as slow as its slowest statement. Find it first, then analyse that statement's plan.

```
-- Statement-level cost inside one procedure (from the plan cache)
SELECT TOP 20 qs.execution_count, qs.total_elapsed_time / 1000 AS total_ms, qs.total_elapsed_time /
    qs.execution_count / 1000 AS avg_ms,
       qs.total_logical_reads / qs.execution_count AS avg_reads,
       SUBSTRING(st.text, qs.statement_start_offset / 2 + 1,
                 (CASE qs.statement_end_offset WHEN -1 THEN DATALENGTH(st.text) ELSE
                     qs.statement_end_offset END - qs.statement_start_offset) / 2 + 1) AS
                     statement_text,
       qs.statement_start_offset
FROM sys.dm_exec_query_stats qs CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
WHERE st.objectid = OBJECT_ID('dbo.GetCustomerOrders') AND st.dbid = DB_ID()
ORDER BY qs.total_elapsed_time DESC;
```

- With **Include Actual Execution Plan**, one plan shows every statement of the procedure with its relative cost, but check the *actual* time and rows, not only cost percentages.
- Look at temp tables created then queried (recompiles), dynamic SQL (separate plans), table variables (estimates), cursors and loops, and scalar functions inside statements.
- For long procedures capture `sp_statement_completed` Extended Events to see per-statement duration in production.

16.7 Slow ETL and bulk loads

Problem	Fix
Fully logged inserts	Use minimal logging conditions (appropriate recovery model, TABLOCK, empty or heap/columnstore targets, supported patterns)
Indexes maintained row by row	Load into a staging heap, then build indexes; or disable nonclustered indexes during load
Tiny batches / row-by-row inserts	Bulk APIs, batch sizes of thousands of rows, table-valued parameters
Foreign keys and triggers during load	Disable or validate afterwards (WITH CHECK)
Single-threaded load	Parallel loads into partitions or separate staging tables
tempdb/log pressure	Size tempdb and log for the load; batch the work

16.8 Slow in Azure SQL Database and Managed Instance

- Queries run inside **resource limits** of the service tier (CPU, memory, data and log I/O, workers, tempdb). Hitting a limit shows as waits and throttling, not errors.
- Look at **resource usage** and **log rate** waits (LOG_RATE_GOVERNOR and related), then scale the tier or reduce work.
- Query Store is on by default and is the main tool; automatic tuning can force good plans.
- **Network and connectivity** (gateway vs redirect, region distance) affect chatty applications.

```
-- Azure SQL Database: resource consumption vs limits in the last hour (15-second samples)
SELECT TOP 20 end_time, avg_cpu_percent, avg_data_io_percent, avg_log_write_percent,
    avg_memory_usage_percent, max_worker_percent, max_session_percent
FROM sys.dm_db_resource_stats ORDER BY end_time DESC;
```

16.9 Slow on a readable secondary or with Availability Groups

- **Redo lag** makes secondaries return old data and can block queries (versioning waits). Check redo_queue_size in sys.dm_hadr_database_replica_states.
- Heavy reporting on a secondary competes with redo for CPU and I/O; consider a separate replica.
- Statistics created on the secondary are temporary; plans may differ from the primary. Query Store on secondaries is available in newer versions.
- Synchronous commit adds commit latency (HADR_SYNC_COMMIT, WRITELOG): measure before blaming the query.

17. Worked Case Studies

Seven realistic cases, each walked through the method: evidence, diagnosis, fix and expected result. Numbers are illustrative.

17.1 Fast in SSMS, slow in the application

Symptom	A procedure returns in 0.2 s in SSMS but takes 40 s from the web application.
Evidence	Two plan cache entries for the procedure with different set_options (ARITHABORT ON vs OFF). The app's plan was compiled for a rare customer (12 rows) and uses Nested Loops with key lookups; the heavy customer returns 2 million rows.
Diagnosis	Parameter sniffing in the application's cached plan; SSMS only looked fast because it compiled its own plan for the test value.
Fix	Reproduce with SET ARITHABORT OFF and the heavy value; add a covering index and a Query Store RECOMPILE hint (or PSP at compatibility 160).
Result	Heavy customers use a scan plan (4 s), small customers keep seeks; variation disappears.

17.2 Sudden regression after a statistics update

Symptom	A dashboard query went from 2 s to 60 s overnight.
Evidence	Query Store: plan changed at 02:10 when auto-update statistics fired. New plan uses Hash Match with a scan; estimated 100 rows, actual 3 million (stale histogram on an ascending date column).
Diagnosis	Plan regression caused by a different plan and a poor estimate.
Fix	Force the previous plan in Query Store, then update statistics with FULLSCAN on the date column and schedule frequent updates; unforce after verification.
Result	Back to 2 s immediately; stable after the statistics fix.

17.3 Key lookup storm

Symptom	An order search takes 12 s for popular products.
Evidence	Index Seek on ProductID plus Key Lookup with 480,000 executions; PAGEIOLATCH_SH waits; 1.9 million logical reads.
Diagnosis	Non-covering index chosen for a wide range; each row costs a random read.
Fix	CREATE INDEX ... (ProductID) INCLUDE (OrderID, OrderDate, Quantity, UnitPrice).
Result	Seek only, 14,000 reads, 0.3 s.

17.4 Spills because of a table variable

Symptom	A reporting procedure sometimes takes minutes and thrashes tempdb.
Evidence	Hash Warning (spill level 2) and Sort Warning; estimated rows 1, actual 4 million on a table variable; IO_COMPLETION waits.
Diagnosis	No statistics on the table variable gave a 1-row estimate and tiny memory grants.
Fix	Replace with a #temp table (with an index) and update statistics after loading.
Result	Correct estimate, no spills, 25 s down to 5 s and stable.

17.5 Blocking from a forgotten transaction

Symptom	Application timeouts every afternoon.
Evidence	40 sessions in LCK_M_S/LCK_M_U behind one session; head blocker sleeping for 25 minutes with an open transaction from a batch service.
Diagnosis	The service opened a transaction, then waited for an external API before committing.
Fix	Commit before the external call; add timeouts; enable RCSI so readers no longer block on writers.
Result	No blocking pile-ups; reader latency back to normal.

17.6 ORM nvarchar parameters and scans

Symptom	Account lookup takes 6 s on a 30-million-row table with an index on AccountNumber.
Evidence	Clustered Index Scan; CONVERT_IMPLICIT(nvarchar(4000), AccountNumber) in the predicate; column is varchar(20).
Diagnosis	The ORM sends Unicode parameters; data type precedence converts the column.
Fix	Configure the ORM to use varchar(20) parameters for this property (or cast the parameter).
Result	Index Seek, a handful of reads, 2 ms.

17.7 tempdb allocation contention

Symptom	Throughput drops at peak; CPU is low.
Evidence	PAGELATCH_UP/EX waits on tempdb pages 2:1:1 and 2:1:3; one tempdb data file; thousands of temp tables created per second.
Diagnosis	Allocation page contention in tempdb.
Fix	Add equally sized tempdb data files (8); reuse temp tables where possible; consider memory-optimized tempdb metadata on SQL Server 2019+.
Result	PAGELATCH waits disappear; throughput recovers.

18. Prevention and Monitoring

The best slow-query fix is catching the problem before users do. A small amount of monitoring and discipline prevents most surprises.

18.1 Turn on and tune the basics

- **Query Store** on every database with sensible capture mode and retention; review regressions weekly.
- **Statistics** maintenance suited to data change rates; extra updates for volatile large tables.
- **Extended Events system_health** (default) for deadlocks and errors; a small custom session for statements over a threshold.
- **Backups, CHECKDB and index maintenance** scheduled apart from peaks, and measured.
- **Correct server settings**: max server memory, MAXDOP, cost threshold, tempdb files, instant file initialisation, power plan.

18.2 What to alert on

Condition	Why
Blocking longer than a few minutes; head blocker with long open transaction	Prevents pile-ups
Deadlocks per hour above baseline	Application or index design issue
CPU sustained high; runnable tasks > 0 for minutes	CPU pressure
Memory grants pending > 0; RESOURCE_SEMAPHORE waits	Grant queue
File read/write latency above thresholds	Storage degradation
Log file usage high, log_reuse_wait_desc stuck, tempdb nearly full	Prevents outages
Queries running longer than N x their normal duration	Early warning of regressions
Plan regressions and failed plan forcing in Query Store	Plan stability
Job failures or jobs exceeding normal duration	Maintenance health
AG redo/send queue growth; HADR_SYNC_COMMIT rising	HA latency

18.3 Developer checklist (prevent slow queries at design time)

Done?	Check
[ ]	Predicates are SARGable and parameter types match column types
[ ]	Joins have predicates and supporting indexes; foreign keys are indexed
[ ]	Only needed columns and rows are returned; paging for lists
[ ]	Set-based code instead of cursors and loops; batches for large DML
[ ]	No scalar UDFs or multi-statement TVFs in hot paths; temp tables instead of table variables for larger sets
[ ]	Transactions are short; no user interaction inside them; retries for deadlocks
[ ]	Tested with production-sized, production-skewed data and with the worst-case parameters
[ ]	Execution plan reviewed (estimates vs actuals, scans, lookups, spills) before release
[ ]	Index and write-cost impact considered; unused indexes cleaned up
[ ]	Timeouts and error handling configured in the application

18.4 Upgrades and releases

- Keep Query Store history across releases; compare plans before and after each deployment.
- Change compatibility level separately from other changes so regressions have one cause.
- Have a documented rollback: previous code, forced plans, hints and configuration values.

Appendix A: Extra Diagnostic Scripts

```
-- A1. Queries whose average duration in the last 24 h is more than twice their 14-day history (Query
    Store regressions)
WITH recent AS (
  SELECT p.query_id, SUM(rs.avg_duration * rs.count_executions) / NULLIF(SUM(rs.count_executions), 0)
      AS avg_dur, SUM(rs.count_executions) AS execs
  FROM sys.query_store_runtime_stats rs JOIN sys.query_store_plan p ON p.plan_id = rs.plan_id
  JOIN sys.query_store_runtime_stats_interval i ON i.runtime_stats_interval_id =
      rs.runtime_stats_interval_id
  WHERE i.start_time >= DATEADD(HOUR, -24, SYSUTCDATETIME()) GROUP BY p.query_id),
hist AS (
  SELECT p.query_id, SUM(rs.avg_duration * rs.count_executions) / NULLIF(SUM(rs.count_executions), 0)
      AS avg_dur
  FROM sys.query_store_runtime_stats rs JOIN sys.query_store_plan p ON p.plan_id = rs.plan_id
  JOIN sys.query_store_runtime_stats_interval i ON i.runtime_stats_interval_id =
      rs.runtime_stats_interval_id
  WHERE i.start_time >= DATEADD(DAY, -14, SYSUTCDATETIME()) AND i.start_time < DATEADD(HOUR, -24,
      SYSUTCDATETIME()) GROUP BY p.query_id)
SELECT r.query_id, r.execs, r.avg_dur / 1000.0 AS recent_ms, h.avg_dur / 1000.0 AS history_ms,
    r.avg_dur / NULLIF(h.avg_dur, 0) AS ratio
FROM recent r JOIN hist h ON h.query_id = r.query_id WHERE r.avg_dur > 2 * h.avg_dur AND r.execs > 10
    ORDER BY ratio DESC;
```

```
-- A2. Database file sizes, growth settings and free space
SELECT DB_NAME(database_id) AS db, name, type_desc, size * 8 / 1024 AS size_mb,
       CASE is_percent_growth WHEN 1 THEN CAST(growth AS varchar(10)) + ' %' ELSE CAST(growth * 8 /
           1024 AS varchar(10)) + ' MB' END AS growth_setting,
       CASE max_size WHEN -1 THEN 'unlimited' ELSE CAST(max_size * 8 / 1024 AS varchar(20)) + ' MB' END
           AS max_size
FROM sys.master_files ORDER BY database_id, type_desc;

-- A3. Largest tables by size and rows (where do the reads come from?)
SELECT TOP 20 OBJECT_NAME(ps.object_id) AS tbl, SUM(ps.row_count) AS rows, SUM(ps.used_page_count) * 8
    / 1024 AS used_mb
FROM sys.dm_db_partition_stats ps WHERE ps.index_id IN (0, 1) AND OBJECTPROPERTY(ps.object_id,
    'IsUserTable') = 1
GROUP BY ps.object_id ORDER BY used_mb DESC;
```

```
-- A4. Plans with implicit conversions that affect seeks or estimates
SELECT TOP 20 qs.execution_count, qs.total_worker_time / 1000 AS total_cpu_ms, qp.query_plan
FROM sys.dm_exec_query_stats qs CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle) qp
WHERE CAST(qp.query_plan AS nvarchar(max)) LIKE '%PlanAffectingConvert%' ORDER BY qs.total_worker_time
    DESC;

-- A5. Plans with spills (Sort/Hash warnings) in the cache
SELECT TOP 20 qs.execution_count, qs.total_worker_time / 1000 AS total_cpu_ms, qp.query_plan
FROM sys.dm_exec_query_stats qs CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle) qp
WHERE CAST(qp.query_plan AS nvarchar(max)) LIKE '%SpillToTempDb%' ORDER BY qs.total_worker_time DESC;

-- A6. Currently running queries using the most tempdb and memory grant
SELECT r.session_id, r.granted_query_memory * 8 / 1024 AS grant_mb, r.wait_type, r.total_elapsed_time /
    1000 AS elapsed_s
FROM sys.dm_exec_requests r WHERE r.granted_query_memory > 0 ORDER BY r.granted_query_memory DESC;
```

Appendix B: One-Page Slow Query Checklist

Done?	Step	What to record
[ ]	Define the problem: what, how slow, since when, who, what changed	Ticket notes
[ ]	60-second health check: running requests, blocking, waits, file latency, memory grants	Screenshots/outputs

Done?	Step	What to record
[ ]	Decide the path: server-wide, one query, or only in the app	Decision and evidence
[ ]	Find the query by impact: Query Store / DMVs / XEvents (frequency x cost)	query_id, query_hash
[ ]	Reproduce with the same parameters, SET options and data volume	Parameters used
[ ]	Capture actual plan, STATISTICS IO and TIME, waits	Saved .sqlplan, numbers
[ ]	Read the plan: estimates vs actuals, scans, lookups, joins, spills, parallelism, warnings	First point where estimate diverges
[ ]	Identify the cause: statistics, sniffing, index, code pattern, blocking, resource, client	One sentence cause
[ ]	Choose the smallest safe fix; test off production; one change at a time	Change and rollback
[ ]	Verify with the same measurements; check side effects	Before/after table
[ ]	Deploy, monitor, document, and add alerting or Query Store checks	Runbook entry

Appendix C: Glossary

Term	Meaning
Actual / Estimated plan	Plan with runtime counters / compiled plan with estimates only
Cardinality estimate	Predicted number of rows for an operator
Statistics	Histograms and densities describing data distribution
Seek / Scan	Navigate an index to keys / read the whole index or range
SARGable	A predicate that can use an index seek
Key Lookup	Fetching missing columns from the clustered index per row
Covering index	An index containing all columns a query needs
Parameter sniffing	Compiling a plan using the first parameter values seen, then reusing it
Plan regression	A new plan that performs worse than the previous one
Query Store	Built-in history of queries, plans, runtime stats and waits
Memory grant / spill	Working memory reserved before execution / writing to tempdb when it is too small
Wait type	Reason a thread is waiting (resource, lock, I/O, network and so on)
Signal wait	Time a ready thread waits for a CPU
Blocking / Deadlock	One session waiting on another's lock / two sessions waiting on each other
RCSI	READ_COMMITTED_SNAPSHOT: readers use row versions instead of locks
Head blocker	The session at the root of a blocking chain
ASYNC_NETWORK_IO	Server waiting for the client to consume results
IQP	Intelligent Query Processing features that adapt plans automatically

Appendix D: Where to Learn More

Details change with each SQL Server release. Use Microsoft's documentation for exact behaviour in your build, and community tools carefully.

Topic	Where to look
Query Store and automatic tuning	Microsoft Learn: 'Monitor performance by using the Query Store', 'Automatic tuning'
Plans and operators	Microsoft Learn: 'Showplan logical and physical operators reference'; execution plan overview
Intelligent Query Processing	Microsoft Learn: 'Intelligent query processing in SQL databases'
Wait types	Microsoft Learn: 'sys.dm_os_wait_stats (Transact-SQL)'
Statistics and cardinality estimation	Microsoft Learn: 'Statistics', 'Cardinality Estimation (SQL Server)'
Locking and row versioning	Microsoft Learn: 'Transaction locking and row versioning guide'
Diagnostic tools	Community scripts and tools (sp_WhoIsActive, First Responder Kit) and Microsoft's Performance Dashboard reports